

Б.В. Пашковський

ПРОГРАМУВАННЯ

ЛАБОРАТОРНИЙ ПРАКТИКУМ

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

**Івано-Франківський національний технічний
університет нафти і газу**

Кафедра комп'ютерних систем та мереж

Б.В. Пашковський

ПРОГРАМУВАННЯ

ЛАБОРАТОРНИЙ ПРАКТИКУМ

**Івано-Франківськ
2026**

УДК 004
П 22

Рецензент:

Шекета В.І. доктор технічних наук, професор кафедри інженерії програмного забезпечення

Івано-Франківського національного технічного університету нафти і газу

Рекомендовано методичною радою університету
(протокол № 6/1 від 20.06.2026 р.).

Пашковський Б.В.

П 22. Програмування: лабораторний практикум. Івано-Франківськ:
ІФНТУНГ, 2026. – 88 с.

МВ 02070855- 20516 -2026

Лабораторний практикум містить методичні вказівки до виконання лабораторних робіт та самостійної роботи з дисципліни «Програмування». Для студентів, які навчаються за освітньо-професійною програмою на здобуття ступеня бакалавра за спеціальністю F7 — Комп'ютерна інженерія та F3 — Комп'ютерні науки .

Опис середовищ програмування, приклади коду програм та частина контрольних запитань і завдань згенеровані за допомогою засобів штучного інтелекту, виправлені та верифіковані вручну.

УДК 004

МВ 02070855- 20516 -2026

© Пашковський Б.В.

© ІФНТУНГ, 2026

Відповідальний за видання
завідувач кафедри
комп'ютерних систем і мереж

С. І. Мельничук

Член експертно-рецензійної
комісії університету

В. В. Бандура

Нормоконтролер

Л. Є. Артим

Головний бібліотекар НТБ

Т. С. Макар

Зміст

Передмова	5
Як працювати з посібником	6
Лабораторна робота №1. Середовище та перша програма	8
Лабораторна робота №2. Змінні та лінійні обчислення	15
Лабораторна робота №3. Логіка та розгалуження	20
Лабораторна робота №4. Цикли	27
Лабораторна робота №5. Функції та декомпозиція	33
Лабораторна робота №6. Одновимірні масиви	40
Лабораторна робота №7. Двовимірні масиви.....	45
Лабораторна робота №8. Символи та рядки.....	50
Лабораторна робота №9. Посилання, вказівники та базова динамічна пам'ять	55
Лабораторна робота №10. Текстові файли	70
Додаток А. Встановлення та перевірка середовища розробки.....	77
Додаток Б. Системи числення.....	81
Додаток В. Приклад оформлення лабораторної роботи	84
Технічні джерела	88

Передмова

Цей посібник допоможе зробити перші кроки в C++. Ви почнете з простої програми, яка виводить текст на екран, а потім навчитесь зчитувати дані, виконувати обчислення, перевіряти умови, повторювати дії за допомогою циклів і розділяти програму на функції.

Далі програми працюватимуть із масивами, таблицями, символами, рядками, адресами та пам'яттю. Завершальна лабораторна покаже, як читати дані з текстового файлу і записувати результат у файл.

Кожна лабораторна містить пояснення, приклади та 20 індивідуальних варіантів. Варіант визначає конкретні дані або умову задачі, але спосіб роботи залишається однаковим: розібрати приклад, написати програму, запустити її та пояснити своє рішення.

Попередні знання C++ не потрібні. Усе необхідне вводиться поступово — від найпростішого коду до роботи з файлами. Додаток А допоможе налаштувати компілятор, а Додаток Б пояснить потрібні системи числення.

Як працювати з посібником

Для кожної лабораторної дотримуйтеся такої послідовності:

1. Прочитайте тему й вступ;
2. Розберіть приклади та виконайте міні-вправи;
3. Визначте свій варіант;
4. Напишіть і скомпілюйте програму;
5. Оформіть файл програми .cpp і звіт у форматі DOCX або PDF;
6. Підготуйтеся пояснити рішення на захисті.

Звіт і назви файлів

Звіт має містити постановку, код як текст і висновок. Називайте файл програми так: LabNN_Surname_Name_Group_Variant.cpp, а файл звіту — LabNN_Surname_Name_Group_Variant.docx

або LabNN_Surname_Name_Group_Variant.pdf. Тут NN — двозначний номер поточної лабораторної роботи: для лабораторної №1 використовуйте Lab01, для №2 — Lab02, ..., для №10 — Lab10; Surname, Name, Group і Variant замініть власними даними.

Приклади для варіанта 3 студентки Олени Петренко з групи КН-11: файл програми для лабораторної №1 — Lab01_Petrenko_Olena_KN-11_03.cpp; файл звіту — Lab01_Petrenko_Olena_KN-11_03.docx або Lab01_Petrenko_Olena_KN-11_03.pdf.

Захист

На захисті потрібно пояснити алгоритм і ключові конструкції, пройти фрагмент коду вручну, передбачити результат для нових даних і внести невелику зміну за завданням викладача.

Критерії оцінювання

Конкретні критерії та шкалу оцінювання визначає викладач відповідно до робочої програми дисципліни.

- Код і відповідність умові: програма компілюється, розв'язує свій варіант і дотримується обмежень теми.
- Пояснення і звіт: є зрозумілий опис рішення, читабельний текст програми та висновок.
- Захист: студент самостійно пояснює рішення, прогнозує результат і виконує коротку зміну.

- Організація подання: подано всі потрібні файли у правильному форматі та з правильними назвами.

Академічна доброчесність і генеративний ШІ

Використання генеративних систем ШІ регулюється правилами закладу та конкретної дисципліни. Застосовувати ШІ можна лише в межах, прямо дозволених цими правилами; можливі допоміжні приклади — пояснення повідомлення компілятора, пошук ідей або мовне редагування, якщо це дозволено викладачем. Не можна делегувати системі виконання роботи та подавати результат, якого студент не перевінив і не може пояснити. Якщо правила дисципліни вимагають розкриття використання ШІ, у звіті зазначте інструмент, мету й частину роботи, для якої його використано. Студент відповідає за коректність програми, звіту та дотримання правил.

Плагіат, подання чужого або згенерованого коду як власного, приховування використання ШІ всупереч правилам дисципліни чи використання ШІ за межами дозволеного розглядають за правилами закладу; захист є обов'язковим.

Оцінювання

- Високий рівень: програма відповідає умові, коректно компілюється, а студент може пояснити ключові рішення.
- Достатній рівень: основну задачу виконано, але окремі пояснення, оформлення або частини рішення потребують уточнення.
- Потребує доопрацювання: суттєва частина умови не виконана або подане рішення не дає змоги підтвердити самостійну роботу.
- Підсумкову оцінку викладач визначає за робочою програмою дисципліни з урахуванням коду, звіту, пояснення та захисту.

Лабораторна робота №1

Середовище та перша програма

1. Мета роботи

Навчитися створювати найпростішу програму C++, компілювати її, запускати та перевіряти послідовність текстового виведення.

Коли ми відкриваємо редактор, перед нами ще немає готової програми: є лише текст, який потрібно зберегти у файлі. Щоб цей текст став програмою, його має опрацювати компілятор. Після успішної компіляції з'являється результат, який можна запустити й перевірити. У цій роботі ви пройдете повний цикл: створення `.cpp`-файлу, редагування, компіляція, запуск і виправлення простої помилки. Змінні, введення, обчислення, умови та цикли з'являться в наступних роботах.

2. Очікувані результати

Після виконання роботи ви зможете:

1. Розрізняти файл `.cpp`, редактор, IDE, компілятор і запущену програму;
2. Створювати програму з функцією `main` та виводити текст через `std::cout`;
3. Передбачати порядок кількох повідомлень;
4. Знаходити прості синтаксичні помилки;
5. Повторювати цикл редагування, компіляції та запуску після зміни коду.

3. Що потрібно знати перед початком

Попередні знання з програмування не потрібні. Потрібні лише редактор тексту та встановлений компілятор C++ або IDE з компілятором. Інструкції для перевірки середовища наведено в додатку А.

4. Теоретичні відомості

Вихідний файл, компілятор і запуск

Вихідний код — це текст програми, який зберігають у файлі з розширенням `.cpp`. Редактор дає змогу змінювати цей текст. Компілятор перевіряє його та створює програму, яку можна запустити. IDE поєднує редактор і команди роботи з проєктом, але сама не замінює компілятор.

Після кожної зміни потрібно зберегти файл, скомпілювати його та запустити новий результат. Якщо запустити старий виконуваний файл, на екрані можна побачити старий результат.

Мінімальна програма

Перед фрагментом розберімо його оболонку. `#include <iostream>` підключає опис засобів стандартного введення-виведення. `int main()` оголошує головну функцію: `int` задає цілий тип її коду завершення, `main` є спеціальним ім'ям, а `()` показують, що в цій простій версії функція не має параметрів. Фігурні дужки `{ }` обмежують тіло функції. У рядку `std::cout << "текст"` повне ім'я `std::cout` позначає стандартний потік виведення, а `<<` передає йому текстовий літерал у подвійних лапках. `\n` переходить на новий рядок, крапка з комою `;` завершує інструкцію, а `return 0;` завершує `main` і повертає код успіху.

```
#include <iostream>

int main()
{
    std::cout << "Hello!\n";
    return 0;
}
```

Результат:

```
Hello!
```

`#include <iostream>` підключає засоби виведення. Виконання починається з `main`. Фігурні дужки позначають тіло функції, `std::cout` виводить текст, `\n` переходить на новий рядок, а крапка з комою завершує інструкцію.

Коротке пояснення синтаксису першої програми:

- `int` перед `main` означає тип коду завершення: головна функція повертає ціле число. Типи даних детальніше розглянемо в лабораторній роботі №2;
- `return 0;` завершує `main` і повертає операційній системі код успіху. Повернення значень і функції детальніше розглянемо в лабораторній роботі №5;
- `std::` є частиною повного імені об'єкта стандартної бібліотеки;
- `<<` у цьому прикладі передає текст до `std::cout`;
- Текст у подвійних лапках є текстовим літералом.

Послідовне виконання

```
#include <iostream>

int main()
{
    std::cout << "One\n";
    std::cout << "Two\n";
    std::cout << "Three\n";
    return 0;
}
```

Результат:

```
One
Two
Three
```

Інструкції в `main` виконуються зверху вниз. Якщо переставити два рядки `std::cout`, зміниться й порядок повідомлень.

5. Типові помилки

Помилка	Причина	Як виправити
<code>std::cout</code> не розпізнано	Немає <code>#include <iostream></code>	Додайте заголовок
Помилка біля наступного рядка	Пропущено <code>;</code>	Перевірте попередню інструкцію
Незакритий текст	Немає парної лапки	Додайте другу <code>"</code>
Показано старий результат	Запущено стару версію програми	Збережіть файл і повторіть компіляцію
Файл не обробляється як C++	Неправильне розширення	Перевірте, що файл має назву <code>.cpp</code>

Навмисно неправильний фрагмент:

```
std::cout << "Missing semicolon\n"
```

Після `std::cout` пропущено `;`. Виправлена версія:

```
std::cout << "Missing semicolon\n";
```

6. Міні-вправи

1. Передбачте результат для трьох послідовних `std::cout`.
2. Переставте два повідомлення та запишіть новий результат.
3. Додайте четвертий рядок без змінних і введення.
4. Знайдіть пропущену крапку з комою.
5. Визначте, чи потрібна повторна компіляція після зміни тексту.
6. Поясніть різницю між `.cpp`-файлом і запущеною програмою.

7. Лабораторне завдання

Створіть одну програму в одному `.cpp`-файлі. Вона має вивести рівно п'ять змістових рядків відповідно до вашого варіанта, використати щонайменше три окремі інструкції `std::cout` і хоча б один раз `\n` у текстовому літералі. Після першого успішного запуску змініть зазначений фрагмент, повторіть компіляцію та запуск і зафіксуйте новий результат.

Під час роботи один раз створіть безпечну синтаксичну помилку — пропущену `;`, лапку або фігурну дужку. Покажіть повідомлення компілятора, виправте код і успішно скомпілюйте програму.

Після зміни коду не запускайте старий результат: спочатку збережіть файл і скомпілюйте програму знову.

Не використовуйте змінні, `std::cin`, арифметику, умови, цикли, масиви, `std::string` як тип, посилання, вказівники, файли чи власні функції.

8. Індивідуальні варіанти

У кожній умові виведіть рівно наведені п'ять рядків. Потім виконайте зазначену зміну.

№	Початкові 5 рядків	Зміна після першого запуску
1	Програма: старт Файл: main.cpp Мова: C++ Вивід: текст Стан: готово	Стан: готово → Стан: перший запуск
2	Назва: Hello Рядок: перший Рядок: другий Рядок: третій Результат: готово	Результат: готово → Результат: виконано
3	Крок 1: написати код Крок 2: зберегти файл Крок 3: скомпілювати Крок 4: запустити Крок 5: прочитати вивід	Крок 5: прочитати вивід → Крок 5: перевірити вивід
4	Лабораторна: перша Файл: main.cpp Функція: main Потік: cout Повідомлення: готово	Повідомлення: готово → Повідомлення: початок
5	Меню програми 1 — почати 2 — довідка 3 — завершити Вибір: очікується	Вибір: очікується → Вибір: прийнято

№	Початкові 5 рядків	Зміна після першого запуску
6	Проект: калькулятор Мова: C++ Файл: main.cpp Компілятор: C++17 Стан: створено	Стан: створено → Стан: зібрано
7	Навчальна програма Тема: виведення тексту Дія: compile Дія: run Підсумок: готово	Підсумок: готово → Підсумок: перевірено
8	Картка програми Автор: студент Версія: 1 Режим: консоль Стан: чернетка	Стан: чернетка → Стан: завершено
9	Послідовність запуску Файл збережено Компіляцію виконано Програму запущено Вивід отримано	Вивід отримано → Вивід прочитано
10	Лабораторна робота Номер: 1 Завдання: п'ять рядків Інструмент: компілятор Готовність: так	Готовність: так → Готовність: підтверджено
11	Файл програми: main.cpp Функція: main Команда: компіляція Команда: запуск Екран: консоль	Екран: консоль → Екран: результат
12	Етап 1: редактор Етап 2: файл Етап 3: компілятор Етап 4: програма Етап 5: вивід	Етап 5: вивід → Етап 5: новий вивід

№	Початкові 5 рядків	Зміна після першого запуску
13	Мініпроект Файл: main.cpp Команда: build Команда: run Результат: видно	Результат: видно → Результат: зрозуміло
14	Консольна програма Вхід: відсутній Вихід: п'ять рядків Помилка: виправлена Стан: готово	Помилка: виправлена → Помилка: перевірена
15	План роботи Створити файл Написати main Додати cout Запустити програму	Запустити програму → Повторити запуск
16	Дані програми Тип: текст Формат: рядки Кількість: п'ять Вивід: консоль	Кількість: п'ять → Кількість: шість
17	Перевірка файлу Назва: main.cpp Розширення: cpp Код: збережено Команда: готова	Код: збережено → Код: змінено
18	Навчальний проєкт Крок: створення Крок: компіляція Крок: запуск Крок: висновок	Крок: висновок → Крок: повторний запуск
19	Сценарій запуску Спочатку: код Потім: компіляція Далі: запуск Наприкінці: вивід	Наприкінці: вивід → Наприкінці: змінений вивід

№	Початкові 5 рядків	Зміна після першого запуску
20	Звітна картка Лабораторна: 1 Варіант: 20 Файл: main.cpp Підсумок: виконано	Підсумок: виконано → Підсумок: пояснено

9. Підготовка до захисту

Підготуйтеся пояснити роль `#include <iostream>`, `main`, `std::cout`, `\n`, фігурних дужок і `return 0;`. Будьте готові передбачити результат після перестановки рядків, знайти пропущену `;` і пояснити, чому після редагування потрібна нова компіляція.

10. Контрольні запитання

1. Чим `.cpp`-файл відрізняється від виконуваної програми?
2. Яку роль виконує компілятор?
3. Чому після зміни коду потрібна повторна компіляція?
4. Для чого потрібен `#include <iostream>`?
5. Що станеться без `;` після `std::cout`?
6. У якому порядку виконуються послідовні інструкції?
7. Чим помилка компіляції відрізняється від проблеми після запуску?
8. Для чого потрібні `{}` у `main`?
9. Чим IDE відрізняється від компілятора?
10. Навіщо читати перше повідомлення компілятора про помилку?

Лабораторна робота №2

Змінні та лінійні обчислення

1. Мета роботи

Навчитися зчитувати числові дані, зберігати їх у змінних, будувати арифметичні вирази та правильно працювати з цілим і дійсним діленням.

Перша програма може вивести наперед записаний текст, але корисна програма повинна працювати з даними. Для цього потрібно десь зберегти введені значення, виконати обчислення й показати результат. У цій роботі ми вводимо типізовані змінні та лінійні арифметичні вирази. Усі інструкції виконуються послідовно: введення → збереження → обчислення → виведення. Умови, цикли, власні функції (крім обов'язкової функції `main`) та масиви з'являться пізніше.

2. Очікувані результати

Після виконання роботи ви зможете:

1. Пояснити різницю між типом, ім'ям і значенням змінної;
2. Оголошувати `int` і `double`, а також зчитувати значення через `std::cin`;
3. Будувати арифметичні вирази з `+`, `-`, `*`, `/` і `%`;
4. Розрізняти цілочисельне та дійсне ділення й за потреби використовувати `static_cast<double>`;
5. Реалізовувати коротку програму за формулою та пояснювати роль `const`.

3. Що потрібно знати перед початком

Потрібні навички з лабораторної №1: створення `.cpp`-файлу, `main`, `std::cout`, компіляція та запуск.

4. Теоретичні відомості

Змінна як іменоване значення

Змінна дає програмі ім'я, під яким можна зберегти значення певного типу:

Спочатку зафіксуємо загальну форму: `тип ім'я = початкове_значення`; Тип визначає, які дані зберігатимуться, ім'я дає до них доступ, а `=` у цьому записі задає початкове значення, а не перевіряє математичну рівність. Створення змінної називаємо оголошенням, а задання першого значення під час створення — ініціалізацією.

```
int age = 18;  
double temperature = 21.5;
```

У першому рядку `int` — тип, `age` — ім'я, `18` — початкове значення. Оголошення створює змінну, ініціалізація одразу задає її початкове значення, а присвоєння змінює вже існуюче значення:

```
int count = 3;
count = 5;
```

Знак `=` у програмі задає значення, а не перевіряє математичну рівність.

Введення

`std::cin` представляє стандартне введення, зазвичай клавіатуру. Оператор `>>` бере наступне значення з потоку й записує його у змінну праворуч; кілька операторів `>>` читають значення послідовно зліва направо.

```
int first;
int second;
std::cin >> first >> second;
```

Якщо ввести `7 3`, у `first` буде 7, а в `second` — 3. Значення передаються у змінні зліва направо.

Арифметичні вирази та дужки

У C++ запис `//` починає однорядковий коментар до кінця рядка. Компілятор ігнорує текст після `//`; у наступному фрагменті числа `14` і `20` є лише примітками з очікуваними результатами, а не частиною обчислення.

```
int firstResult = 2 + 3 * 4;    // 14
int secondResult = (2 + 3) * 4; // 20
```

Множення й ділення виконуються перед додаванням і відніманням. Дужки явно показують потрібне групування.

Цілочисельне та дійсне ділення

Спочатку подивімося на різницю між двома простими виразами:

```
int integerResult = 5 / 2;    // 2
double realResult = 5.0 / 2.0; // 2.5
```

Якщо обидва операнди мають цілий тип, дробова частина не зберігається. Тому `double wrong = 5 / 2;` дає `2.0`, а не `2.5`. Щоб змінити тип обчислення до ділення, використовуйте:

```
int sum = 5;
int count = 2;
double average = static_cast<double>(sum) / count;
```

Тут `sum` і `count` залишаються змінними типу `int`, але перед діленням `sum` тимчасово перетворюється на `double`. Простішим записом буде `sum / 3.0`, якщо дільник завжди дорівнює 3.

Остача та незмінні значення

Оператор `%` повертає остачу від ділення цілих чисел: `17 % 5` дорівнює `2`. `const` позначає значення, яке після ініціалізації не планують змінювати:

```
const int divisor = 7;
int remainder = 20 % divisor;
```

5. Приклади

Приклад 1. Сума і різниця

```
#include <iostream>

int main()
{
    int first;
    int second;
    std::cin >> first >> second;
    int sum = first + second;
    int difference = first - second;
    std::cout << "Sum: " << sum << "\n";
    std::cout << "Difference: " << difference << "\n";
    return 0;
}
```

Введення `8 3` дає результат `Sum: 11` і `Difference: 5`.

Приклад 2. Дійсне середнє

```
#include <iostream>

int main()
{
    int first;
    int second;
    int third;
    std::cin >> first >> second >> third;
    int sum = first + second + third;
    double average = static_cast<double>(sum) / 3;
    std::cout << "Sum: " << sum << "\n";
    std::cout << "Average: " << average << "\n";
    return 0;
}
```

Для введення `4 5 6` результатом є сума `15` і середнє `5`.

6. Типові помилки

- Використання змінної до ініціалізації;
- Очікування `2.5` від виразу `5 / 2`;
- Застосування `%` до `double`;
- Неправильне групування формули без дужок;
- Зміна `const`-значення після ініціалізації;
- Плутання присвоєння `=` з перевіркою умови, яку розглянемо в наступній роботі.

7. Міні-вправи

1. Назвіть тип, ім'я та значення в `double temperature = 21.5`.

- Обчисліть $2 + 3 * 4$ і $(2 + 3) * 4$.
- Поясніть різницю між $5 / 2$ і $5.0 / 2.0$.
- Виправте обчислення середнього двох цілих змінних.
- Обчисліть $23 \% 7$.
- Додайте `const` до незмінного дільника.
- Знайдіть помилку в `value % 2.0`.

8. Лабораторне завдання

Оберіть один номер у таблиці індивідуальних варіантів. У таблиці для кожного варіанта вказано формулу для `integerResult` і значення `divisor`. Виконайте тільки свій варіант.

Користувач вводить три цілі числа `a`, `b`, `c` у межах від 0 до 100. У програмі послідовно:

- Оголосіть змінні `int a`, `int b` і `int c` та зчитайте їх через `std::cin`.
- Обчисліть суму `sum = a + b + c`.
- Створіть константу `const int divisor` і надайте їй значення з таблиці.
- Обчисліть `integerResult` за формулою зі свого рядка таблиці.
- Обчисліть остачу `remainder = sum % divisor`.
- Обчисліть дійсний результат `realResult = static_cast<double>(sum) / divisor`.
- Виведіть три результати з підписами: `integerResult`, `remainder` і `realResult`.

Отже, в одній програмі ви окремо побачите результат формули, остачу від ділення та ділення з дробовою частиною. Значення `divisor` не змінюйте під час роботи: воно задане вашим варіантом.

Для вхідних значень використайте тип `int`, для `realResult` — тип `double`. Обов'язково використайте проміжну змінну `sum`, `const int divisor`, оператор `%` і `static_cast<double>`. Не використовуйте умови, цикли, власні функції, масиви чи рядки.

9. Індивідуальні варіанти

№	Формула для <code>integerResult</code>	Значення <code>divisor</code>
1	<code>a + b * c</code>	3
2	<code>a * b + c</code>	4
3	<code>a * c + b</code>	5
4	<code>(a + b) * c</code>	6
5	<code>(a + c) * b</code>	7
6	<code>(b + c) * a</code>	8
7	<code>a * b - c</code>	9

№	Формула для integerResult	Значення divisor
8	$a * c - b$	10
9	$b * c - a$	3
10	$a + b - c$	4
11	$a - b + c$	5
12	$b + c - a$	6
13	$c + a - b$	7
14	$a * 2 + b$	8
15	$b * 2 + c$	9
16	$c * 2 + a$	10
17	$(a + b) * 2$	3
18	$(b + c) * 2$	4
19	$(a + c) * 2$	5
20	$c + a * 2$	6

10. Підготовка до захисту

Підготуйтеся вручну обчислити результат на нових даних, пояснити різницю `int` і `double`, показати вплив `static_cast<double>`, змінити `divisor`, пояснити `const` і знайти помилку в арифметичному виразі.

11. Контрольні запитання

1. Що таке тип, ім'я та значення змінної?
2. Чим ініціалізація відрізняється від присвоєння?
3. Чому $5 / 2$ дорівнює 2?
4. Як отримати дійсний результат ділення?
5. Для чого потрібен `static_cast<double>`?
6. Що обчислює `%`?
7. Чому `%` не застосовують до `double`?
8. Як дужки змінюють арифметичний вираз?
9. Навіщо потрібен `const`?
10. Яка помилка в `double r = 10 % 3.0`?

Лабораторна робота №3

Логіка та розгалуження

1. Мета роботи

Навчитися будувати логічні умови, перевіряти межі та реалізовувати взаємовиключні гілки програми.

Лінійна програма виконує однакову послідовність інструкцій для кожного набору даних. Але в реальній задачі результат часто залежить від умови: число може бути додатним або від'ємним, оцінка — достатньою або недостатньою, а значення — потрапляти в один із кількох інтервалів. У цій роботі програма навчиться обчислювати логічний вираз і на його основі вибирати одну гілку виконання. Головним інструментом буде `if / else if / else`; `switch` розглянемо лише як коротку варіацію для дискретних значень.

2. Очікувані результати

Після виконання роботи ви зможете:

1. Пояснити, що результат порівняння має тип `bool`, і використовувати основні оператори порівняння;
2. Будувати `if`, `if / else` та ланцюжок `if / else if / else`;
3. Комбінувати умови через `&&`, `||` і `!`;
4. Правильно задавати межі інтервалів і порядок взаємовиключних перевірок;
5. Відрізнати присвоєння `=` від порівняння `==` та читати простий додатковий приклад із `switch`.

3. Що потрібно знати перед початком

Потрібні змінні, введення, виведення, арифметика, `%`, `static_cast` і послідовне виконання з лабораторної №2. Цикли, власні функції (крім обов'язкової функції `main`), масиви, рядки, посилання, вказівники та файли в цій роботі не використовуються.

4. Теоретичні відомості

Порівняння і `bool`

Порівняння саме по собі не вибирає дію. Воно обчислює логічне значення `true` або `false`:

`bool` — тип для двох логічних значень. Оператори `==` і `!=` означають «дорівнює» та «не дорівнює», `<` і `<=` — «менше» та «менше або дорівнює», а `>` і `>=` —

«більше» та «більше або дорівнює». Оператор `=` має іншу роль: він присвоює значення змінній.

```
int x = 5;  
bool isFive = (x == 5);  
bool isLarge = (x > 10);
```

`x = 5` змінює значення `x`, а `x == 5` перевіряє рівність. Це одна з найважливіших відмінностей у цій роботі.

if і if / else

`if` спочатку обчислює умову в круглих дужках. Якщо результат `true`, виконується тіло першої гілки; у конструкції `if / else` за результатом `false` виконується рівно одна альтернативна гілка. Фігурні дужки позначають тіло кожної гілки.

```
if (value < 0)  
{  
    std::cout << "Negative\n";  
}  
else  
{  
    std::cout << "Non-negative\n";  
}
```

Якщо умова істинна, виконується перша гілка; інакше — друга. Фігурні дужки в цій книзі використовуємо завжди, навіть для однієї інструкції.

Ланцюжок умов

Умови `else if` перевіряються зверху вниз. Виконується перша істинна гілка, після чого решта ланцюжка пропускається:

```
if (score >= 90)  
{  
    std::cout << "Excellent\n";  
}  
else if (score >= 60)  
{  
    std::cout << "Pass\n";  
}  
else  
{  
    std::cout << "Review\n";  
}
```

Сильнішу, вузку умову потрібно ставити раніше за ширшу. Інакше частина значень стане недосяжною для наступної гілки.

Складені умови та межі

`A && B` істинне, коли істинні обидві частини; `A || B` — коли істинна хоча б одна; `!A` заперечує результат `A`. Замкнений інтервал від 10 до 20 записують так:

```
x >= 10 && x <= 20
```

Запис `10 <= x <= 20` не є подвійною нерівністю в C++. Перевіряйте граничні значення: 9, 10, 20 і 21.

Коротко про switch

`switch` зручний, коли вибір залежить від одного дискретного коду. Після `switch (code)` програма порівнює значення з мітками `case`; `default` задає гілку для всіх інших значень. `break` завершує цей `switch`, щоб після виконаної гілки не перейти до наступної мітки. Тепер прочитаємо короткий приклад:

```
switch (code)
{
case 1:
    std::cout << "Start\n";
    break;
case 2:
    std::cout << "Stop\n";
    break;
default:
    std::cout << "Unknown\n";
    break;
}
```

У головному індивідуальному завданні використовуйте `if / else if / else`. `switch` потрібен лише для розуміння іншого способу вибору.

5. Приклади

Приклад 1. Парність числа

```
int value;
std::cin >> value;
if (value % 2 == 0)
{
    std::cout << "Even\n";
}
else
{
    std::cout << "Odd\n";
}
```

Для 8 результат `Even`, для 5 — `Odd`.

Приклад 2. Чотири інтервали

```
if (value < 0)
{
    std::cout << "Below\n";
}
else if (value <= 10)
{
    std::cout << "Low\n";
}
else if (value <= 20)
{
    std::cout << "Middle\n";
}
else
{
    std::cout << "High\n";
}
```

Приклад поведінки на граничних значеннях:

```
-1 → Below  
0 → Low  
10 → Low  
11 → Middle  
20 → Middle  
21 → High
```

Приклад 3. Дві умови через &&

```
if (first >= 0 && first <= 100 && second >= 0 && second <= 100)  
{  
    std::cout << "Both valid\n";  
}  
else  
{  
    std::cout << "At least one invalid\n";  
}
```

Введення 40 100 дає Both valid, а 40 101 — At least one invalid.

6. Типові помилки

1. = замість == змінює значення замість перевірки.
2. Умова $x < 10$ не охоплює $x == 10$.
3. Запис $10 \leq x \leq 20$ не задає потрібний інтервал.
4. Широка умова перед вузькою робить наступну гілку недосяжною.
5. Два незалежні if можуть виконати обидві дії, а ланцюжок else if обирає одну.
6. && і || мають різний логічний зміст.
7. Крапка з комою після if (condition); завершує порожню інструкцію.
8. У switch відсутній break, тому виконання може перейти до наступного case.

7. Міні-вправи

1. Для $x = 7$ визначте результати $x < 10$, $x == 7$, $x != 7$.
2. Передбачте гілку для $x = 10$ у прикладі з інтервалами.
3. виправте `if (x = 0)`.
4. Запишіть умову «обидва значення від 0 до 100» через &&.
5. Замініть $<$ на $\leq$ і поясніть зміну для граничного значення.
6. Виберіть && або || для умови «число менше 0 або більше 100».
7. Знайдіть пропущене значення в парах умов $x < 0$ і $x > 0$.
8. Поясніть, коли для коду зручніше використати switch.

8. Лабораторне завдання

Програма читає два цілі значення типу `int` і класифікує їх в одну з чотирьох взаємовиключних категорій вашого варіанта. Кожне значення вводьте в межах `[-100000, 100000]`; для варіанта 6 змінна `hasCard` додатково має бути лише 0 або 1.

1. Реалізуйте один ланцюжок `if / else if / else`, використайте оператори порівняння та щонайменше одну складену умову через && або ||.

Використайте два введені значення та виведіть одну підсумкову категорію. Реалізуйте всі категорії, наведені у своєму варіанті, зокрема обробку граничних значень. Не використовуйте цикли, власні функції (крім обов'язкової функції `main`), масиви, рядки чи `switch` в основному завданні.

9. Індивідуальні варіанти

№	Вхідні дані й категорії у порядку перевірки	Приклад input → output
1	score, attempts: invalid якщо <code>score < 0 score > 100 attempts < 0</code> ; excellent якщо <code>score >= 90 && attempts <= 2</code> ; passed якщо <code>score >= 60 && attempts <= 5</code> ; інакше <code>needs_review</code>	95 2 → excellent
2	temperature, humidity: invalid якщо <code>temperature < -50 temperature > 60 humidity < 0 humidity > 100</code> ; cold якщо <code>temperature < 10</code> ; comfortable якщо <code>temperature <= 25 && humidity <= 70</code> ; інакше <code>hot_or_humid</code>	20 60 → comfortable
3	balance, purchase: invalid якщо <code>balance < 0 purchase < 0</code> ; free якщо <code>purchase == 0</code> ; affordable якщо <code>purchase <= balance</code> ; інакше <code>not_affordable</code>	100 70 → affordable
4	speed, limit: invalid якщо <code>speed < 0 limit <= 0</code> ; slow якщо <code>speed < limit</code> ; at_limit якщо <code>speed == limit</code> ; інакше <code>over_limit</code>	60 60 → at_limit
5	length, width: invalid якщо <code>length <= 0 width <= 0</code> ; square якщо <code>length == width</code> ; wide якщо <code>width > length</code> ; інакше <code>tall</code>	4 6 → wide
6	age, hasCard (0/1): invalid якщо <code>age < 0 hasCard < 0 hasCard > 1</code> ; child якщо <code>age < 14</code> ; member якщо <code>hasCard == 1 && age >= 14</code> ; інакше <code>visitor</code>	20 1 → member
7	level, minutes: invalid якщо <code>level < 0 minutes < 0</code> ; beginner якщо <code>level <= 2 && minutes <= 30</code> ; regular якщо <code>level <= 5 minutes <= 60</code> ; інакше <code>advanced</code>	4 20 → regular
8	rpm, temperature: stopped якщо <code>rpm == 0</code> ; low якщо <code>rpm > 0 && temperature < 40</code> ; normal якщо <code>rpm > 0 && temperature <= 70</code> ; інакше <code>hot</code>	1000 70 → normal

№	Вхідні дані й категорії у порядку перевірки	Приклад input → output
9	freeSpace, fileSize: invalid якщо будь-яке < 0; not_enough_space якщо fileSize > freeSpace; low_reserve якщо freeSpace - fileSize < 10; інакше enough_space	20 15 → low_reserve
10	score, absences: invalid якщо score < 0 score > 100 absences < 0; excellent якщо score >= 95 && absences <= 1; passed якщо score >= 70 && absences <= 3; інакше needs_review	95 1 → excellent
11	years, failures: invalid якщо будь-яке < 0; new_stable якщо years <= 2 && failures <= 1; usable якщо years <= 5 && failures <= 3; інакше service_needed	4 2 → usable
12	responseMs, errorPercent: invalid якщо responseMs < 0 errorPercent < 0 errorPercent > 100; good якщо responseMs <= 200 && errorPercent <= 1; acceptable якщо responseMs <= 500 && errorPercent <= 5; інакше poor	200 1 → good
13	temperature, minutes: invalid якщо minutes < 0; too_hot якщо temperature > 70; ready якщо temperature >= 40 && temperature <= 70 && minutes <= 30; інакше cooled	70 30 → ready
14	messages, badPercent: invalid якщо messages < 0 badPercent < 0 badPercent > 100; empty якщо messages == 0; clean якщо messages > 0 && badPercent <= 2; інакше noisy	10 2 → clean
15	voltage, current: invalid якщо будь-яке < 0; low якщо voltage < 200; normal якщо voltage <= 240 && current <= 10; інакше warning	220 10 → normal
16	total, damaged: invalid якщо total < 0 damaged < 0 damaged > total; empty якщо total == 0; good_batch якщо damaged <= 2; інакше review_batch	10 2 → good_batch
17	temperature, memory: invalid якщо memory < 0 memory > 100; normal якщо temperature <= 70 && memory <= 80; memory_warning якщо temperature <= 70 && memory > 80; інакше temperature_warning	70 81 → memory_warning

№	Вхідні дані й категорії у порядку перевірки	Приклад input → output
18	distance, charge: invalid якщо distance < 0 charge < 0 charge > 100; cannot_start якщо charge < 20; short_route якщо charge >= 20 && distance <= 10; інакше long_route	10 20 → short_route
19	attempts, correct: invalid якщо attempts < 0 correct < 0 correct > attempts; no_attempts якщо attempts == 0; successful якщо correct * 2 >= attempts; інакше needs_practice	4 2 → successful
20	timeMs, memoryMb: invalid якщо будь-яке < 0; efficient якщо timeMs <= 1000 && memoryMb <= 256; acceptable якщо timeMs <= 3000 && memoryMb <= 512; інакше heavy	1000 256 → efficient

Для кожного варіанта врахуйте значення точно на межі. Одна пара input має давати лише одну категорію.

10. Підготовка до захисту

Підготуйтеся пройти програму для нового введення, змінити < на <=, замінити && на ||, пояснити порядок else if, знайти помилку в /=== і пояснити поведінку на граничних значеннях.

11. Контрольні запитання

1. Чим = відрізняється від ==?
2. Який тип має результат порівняння?
3. Коли виконується тіло if?
4. Як перевіряються гілки else if?
5. Чому порядок умов важливий?
6. Чим && відрізняється від ||?
7. Який сенс має !?
8. Чому 10 <= x <= 20 некоректний запис інтервалу?
9. Що змінює < проти <=?
10. Коли зручний switch?

Лабораторна робота №4

Цикли

1. Мета роботи

Навчитися повторювати дії відому кількість разів або доти, доки виконується умова, не допускаючи пропущених чи зайвих ітерацій.

Якщо потрібно обробити числа від 1 до 100, копіювати одну й ту саму інструкцію сто разів незручно й небезпечно. Цикл описує повторення компактно: він зберігає поточний стан, перевіряє умову, виконує тіло та переходить до наступної ітерації. У цій роботі спочатку розглянемо `for` для відомого діапазону, потім навчимося накопичувати суму й лічильник, а далі порівняємо `while` і `do...while`. `break` і `continue` будуть короткими інструментами для окремих випадків, а не центром основного завдання.

2. Очікувані результати

Після виконання роботи ви зможете:

1. Пояснити, що таке ітерація, і написати `for` для цілочисельного діапазону;
2. Визначати кількість повторень;
3. Накопичувати суму та рахувати вибрані значення;
4. Порівнювати `while` і `do...while` та писати цикл із гарантованим завершенням;
5. Знаходити помилку межі або очевидний нескінченний цикл; `break` і `continue` розглядати як додаткові інструменти.

3. Що потрібно знати перед початком

Потрібні змінні, арифметика, `%`, порівняння та розгалуження з лабораторних №2–3. Масиви, власні функції, рядки, файли, посилання та вказівники ще не використовуються.

4. Теоретичні відомості

Одна ітерація

Ітерація — одне виконання тіла циклу. Загальна схема така: початковий стан → перевірка умови → тіло → оновлення стану → наступна перевірка.

Цикл `for`

У попередніх прикладах новий рядок записували як `"\n"`. У цій роботі використовуємо той самий уже відомий запис; символні літерали й одинарні

лапки будуть окремо розглянуті в лабораторній роботі №8. У заголовку `for` у круглих дужках послідовно записують початкову дію, умову продовження та оновлення лічильника, розділяючи ці три частини крапками з комою.

```
for (int i = 1; i <= 5; ++i)
{
    std::cout << i << "\n";
}
```

У заголовку є початкове значення `i = 1`, умова `i <= 5` та оновлення `++i`. Тіло виконається для 1, 2, 3, 4 і 5. Варіант `i < 5` дав би лише 1, 2, 3 і 4.

Сума та лічильник

```
int sum = 0;
int count = 0;
for (int i = 1; i <= n; ++i)
{
    sum += i;
    if (i % 3 == 0)
    {
        ++count;
    }
}
```

`sum` накопичує значення, а `count` рахує лише ті значення, які відповідають умові. Це різні ролі, тому для них використовують різні змінні.

Цикл while

`while` зручний, коли кількість повторень наперед невідома. Умова перевіряється перед тілом:

```
int value;
int sum = 0;
std::cin >> value;
while (value != 0)
{
    sum += value;
    std::cin >> value;
}
```

У цьому шаблоні `value` потрібно оновлювати в тілі. Загалом цикл повинен мати спосіб змінювати стан так, щоб умова могла стати хибною: зміна може відбуватися в тілі, під час читання або в іншій частині перевірки. Якщо стан не змінюється, умова може назавжди залишитися істинною.

Цикл do...while

У `do...while` тіло виконується перед першою перевіркою, тому гарантовано відбувається хоча б одна ітерація:

```
int value;
do
{
    std::cin >> value;
}
while (value < 1 || value > 10);
```

`for` природний для діапазону, `while` — для повторення до події або сигналу, `do...while` — коли введення чи дія повинні відбутися хоча б один раз.

break i continue

`break` зупиняє найближчий цикл. `continue` пропускає залишок поточної ітерації та переходить до наступної. У цій роботі це додаткові інструменти для читання прикладів; основне завдання виконується без них.

5. Приклади

Приклад 1. Сума від 1 до n

Для цього прикладу припускаємо $0 \leq n \leq 10000$ і технічну модель `int` з Додатка А. За цієї умови накопичена сума залишається в діапазоні 32-бітного `int`.

```
#include <iostream>

int main()
{
    int n;
    std::cin >> n;
    int sum = 0;
    for (int i = 1; i <= n; ++i)
    {
        sum += i;
    }
    std::cout << "Sum: " << sum << "\n";
    return 0;
}
```

Для $n = 4$ значення `sum` проходять $0 \rightarrow 1 \rightarrow 3 \rightarrow 6 \rightarrow 10$, тому результат — `Sum: 10`.

Приклад 2. while із нулем-сигналом

Якщо ввести `4 6 0`, цикл додасть 4 і 6, а нуль лише завершить введення. Результат — `Sum: 10`.

Приклад 3. Зворотний відлік

```
for (int i = n; i >= 1; --i)
{
    std::cout << i << "\n";
}
```

Для $n = 3$ результатом будуть 3, 2 і 1.

6. Типові помилки

1. $i < 5$ замість $i \leq 5$ створює помилку межі.
2. Відсутність оновлення у `while` створює нескінченний цикл.
3. `sum` починають із 1 замість нейтрального значення 0.
4. Додатковий `++i` у тілі `for` змінює крок і може пропускати числа.

5. `continue` не завершує весь цикл.
6. `break` завершує найближчий цикл або `switch`, а не будь-яку довільну зовнішню конструкцію.
7. Крапка з комою після заголовка циклу робить його тіло порожнім.

Навмисно неправильний приклад:

```
int i = 1;
while (i <= 5)
{
    std::cout << i << "\n";
}
```

`i` не змінюється, тому умова не стає хибною. Додайте `++i` у тіло циклу.

7. Міні-вправи

1. Назвіть усі значення `i` у `for (int i = 2; i <= 8; i += 2)`.
2. Визначте кількість ітерацій циклу від 0 до 3 включно.
3. Передбачте результат зворотного відліку від 3 до 1.
4. виправте межу, якщо потрібно вивести 1–5 включно.
5. Додайте оновлення до простого `while`.
6. Порівняйте `while (false)` і `do...while` з такою самою умовою.
7. Для проходження від 1 до 100 оберіть **найдоречніший** із `for` і `while` та коротко поясніть вибір.
8. Для введення до сигналу 0 оберіть **найдоречніший** із `for` і `while` та коротко поясніть вибір.
9. Для чисел від 1 до 5 одним циклом `for` відберіть парні значення. Простежте стан після кожної ітерації у вигляді `i | count | sum` для умови `i % 2 == 0`.
Очікуваний кінцевий стан: `count = 2, sum = 6`.

8. Лабораторне завдання

Введіть два цілі числа `a` і `b`, де `1 <= a <= b <= 10000`. Якщо діапазон некоректний, повторіть введення через `do...while`. Одним циклом `for` пройдіть усі числа від `a` до `b` включно. Через умову варіанта виберіть потрібні числа, а в окремих змінних накопичте `count` і `sum`.

Виведіть:

```
Count: <кількість>
Sum: <сума>
```

Якщо вибраних чисел немає, обидва результати мають бути нульовими. Не використовуйте масиви, власні функції (крім обов'язкової функції `main`), вкладені цикли, `std::vector` або бібліотечні алгоритми.

9. Індивідуальні варіанти

№	Умова вибору
1	<code>value % 2 == 0</code>

№	Умова вибору
2	<code>value % 2 != 0</code>
3	<code>value % 3 == 0</code>
4	<code>value % 3 != 0</code>
5	<code>value % 4 == 0</code>
6	<code>value % 5 == 0</code>
7	<code>value % 6 == 2</code>
8	<code>value % 7 == 0</code>
9	<code>value % 3 == 1</code>
10	<code>value % 4 == 1</code>
11	<code>value % 4 == 2</code>
12	<code>value % 5 == 1</code>
13	<code>value % 5 == 2</code>
14	<code>value % 5 == 3</code>
15	<code>value % 2 == 0 && value % 3 == 0</code>
16	<code>value % 2 == 0 && value % 3 != 0</code>
17	<code>value % 2 != 0 && value % 3 == 0</code>
18	<code>value % 3 == 0 value % 5 == 0</code>
19	<code>value % 4 == 0 value % 4 == 1</code>
20	<code>value % 6 == 1 value % 6 == 5</code>

Варіант 18 рахує число лише один раз, навіть якщо воно ділиться і на 3, і на 5: цикл проходить кожне число один раз.

Приклад варіанта

Варіант 1, input 2 7: вибрані 2, 4, 6, тому output:

```
Count: 3
Sum: 12
```

10. Підготовка до захисту

Підготуйтеся пояснити перші три ітерації, змінити `<=` на `<`, знайти причину нескінченного `while`, змінити умову варіанта, пояснити різницю між `count` і `sum`, а також сказати, коли доречні `for`, `while` і `do...while`.

11. Контрольні запитання

1. Які частини має заголовок `for`?
2. Скільки разів виконається заданий цикл?
3. Чим `i < n` відрізняється від `i <= n`?
4. Що станеться без оновлення стану у `while`?
5. Чому `sum` починають із нуля?
6. Чим `count` відрізняється від лічильника ітерацій?
7. Коли використовують `for`, а коли `while`?
8. Чому `do...while` виконується хоча б один раз?
9. Що робить `break`?
10. Що робить `continue`?

Лабораторна робота №5

Функції та декомпозиція

1. Мета роботи

Навчитися розділяти складену задачу на невеликі іменовані функції з параметрами та результатом.

Коли `main` одночасно читає дані, проходить цикл, перевіряє умови й виводить результат, програму важко читати та перевіряти. Функція дає ім'я одній невеликій операції. Тоді в `main` видно загальний план, а деталі заховані за зрозумілими викликами. У цій роботі ми навчимося передавати значення параметрам, повертати результат через `return`, розуміти локальну область видимості та ділити задачу на кілька функцій. Аналіз цифр є коротким прикладом для декомпозиції; перед основним завданням наведено мінімальний шаблон такого аналізу, а нова головна тема — функції та розподіл відповідальності між ними.

2. Очікувані результати

Після виконання роботи ви зможете:

1. Пояснити, навіщо частину задачі виносять у функцію, і відрізнити визначення від виклику;
2. Розрізняти параметр і аргумент та передавати дані за значенням;
3. Повертати результат через `return`;
4. Пояснювати локальну область видимості та використовувати прототип;
5. Розбивати задачу на кілька пов'язаних функцій без дублювання обчислень.

3. Що потрібно знати перед початком

Потрібні змінні, умови та цикли з попередніх лабораторних. Масиви, рядки, посилання, вказівники, рекурсія, класи та STL у цій роботі не використовуються.

4. Теоретичні відомості

Функція як окрема операція

Функція отримує вхідні значення, виконує одну зрозумілу операцію й повертає результат:

```
int square(int value)
{
    return value * value;
}
```

`int` перед назвою — тип результату, `square` — ім'я, `value` — параметр. Під час виклику `square(4)` число 4 є аргументом, а функція повертає 16.

Передавання за значенням

Параметр за значенням є окремим значенням усередині функції. `void` перед назвою функції означає відсутність результату, який функція повертає виклику: це не спеціальне значення, а вказівка, що результату немає. Така функція може виконати дію, але не використовує `return` із результатом.

```
void showChanged(int value)
{
    value = 100;
}
```

Якщо викликати `showChanged(number)`, зміна `value` не змінить `number` у функції, яка здійснила виклик.

`return` і локальна область видимості

`return` передає результат і завершує поточний виклик. Параметри та змінні, створені всередині функції, доступні лише в цій функції. Дані передають через параметри, а результат — через `return`; глобальні змінні в цій роботі не потрібні.

Прототип

Якщо визначення функції розташоване після `main`, перед викликом оголошують прототип:

```
int square(int value);
```

Прототип повідомляє компілятору ім'я функції, тип результату та параметри.

Декомпозиція

Добре поділена програма може мати функцію для характеристики одного числа, функцію для перевірки критерію та функцію для підсумку трьох результатів. Така структура зменшує дублювання й дозволяє перевіряти частини алгоритму окремо.

5. Приклади

Приклад 1. Функція, що повертає число

```
#include <iostream>

int square(int value)
{
    return value * value;
}

int main()
{
    int number;
    std::cin >> number;
    std::cout << square(number) << "\n";
    return 0;
}
```

Введення 4 дає результат 16.

Приклад 2. Ланцюжок функцій

```
int characteristic(int number)
{
    return number % 10;
}

bool qualifies(int number)
{
    return characteristic(number) == 5;
}

int number = 25;
if (qualifies(number))
{
    std::cout << "yes\n";
}
else
{
    std::cout << "no\n";
}
```

`main` викликає `qualifies`, а `qualifies` викликає `characteristic` і перевіряє її результат. Для `number = 25` очікуваний вивід:

```
yes
```

Приклад 3. Функція з локальним циклом

```
int sumDigits(int number)
{
    int sum = 0;
    while (number > 0)
    {
        sum += number % 10;
        number /= 10;
    }
    return sum;
}
```

Для 1234 функція повертає 10. Зміна локального `number` не змінює аргумент у `main`.

Арифметичний аналіз цифр

Для додатного цілого числа поточна остання цифра — `number % 10`; після цього `number /= 10` відкидає її. Повторення доти, доки копія числа не стане 0, дає всі цифри. Щоб отримати першу цифру без рядка, працюйте з копією числа: поки копія не менша за 10, діліть її на 10; останнє значення є першою цифрою. Для 0 першою цифрою вважають 0. Мінімум і максимум цифр починайте з першої отриманої цифри, а не з довільного числа.

```
int firstDigit(int number)
{
    if (number < 0) number = -number;
    while (number >= 10)
    {
        number /= 10;
    }
    return number;
}
```

Наприклад, `firstDigit(582)` повертає 5. Для варіантів 17–20 та інших характеристик використовуйте той самий принцип роботи з копією числа, не перетворюючи його на рядок.

6. Типові помилки

1. Функція з типом `int` не повертає значення.
2. Параметр плутають з аргументом.
3. Очікують, що параметр за значенням змінить початкову змінну.
4. Локальну змінну використовують поза функцією.
5. Функцію викликають до оголошення без прототипу.
6. Передають неправильну кількість аргументів.
7. Дублюють один алгоритм замість повторного виклику функції.
8. Одна функція виконує багато непов'язаних дій.

Навмисно неправильний фрагмент:

```
int square(int value)
{
    value * value;
}
```

Функція має тип `int`, але не повертає результат. Потрібно написати `return value * value;`.

7. Міні-вправи

1. Обчисліть результат виклику `square(6)`.
2. Назвіть параметр і аргумент у `square(number)`.
3. Напишіть функцію, що повертає подвоєне ціле значення.
4. Додайте `return` у функцію `int cube(int value)`.
5. Передбачте, чи зміниться аргумент після зміни параметра за значенням.

6. Знайдіть помилку області видимості у фрагменті:

```
void run()
{
    if (true)
    {
        int local = 7;
    }
    std::cout << local << "\n";
}
```

Назвіть помилковий рядок і поясніть, де закінчується область видимості `local`.

7. У фрагменті нижче додайте перед `run` прототип `int square(int value);`:

```
void run()
{
    std::cout << square(4) << "\n";
}

int square(int value)
{
    return value * value;
}
```

Очікуваний результат виклику `run()` — 16.

8. У фрагменті знайдіть дві однакові за структурою операції та винесіть їх у функцію `void printSquare(int value)`:

```
void run()
{
    int first = 3;
    int second = 5;
    std::cout << first * first << "\n";
    std::cout << second * second << "\n";
}
```

Замініть два рядки виведення на виклики нової функції. Очікуваний вивід: 9, потім 25.

8. Лабораторне завдання

Введіть три додатні цілі числа `a`, `b`, `c`, кожне не більше 999999. Реалізуйте рівно три змістовні функції:

```
int characteristic(int n);
bool qualifies(int n);
int countQualifying(int a, int b, int c);
```

`characteristic` циклом аналізує цифри одного числа. `qualifies` викликає `characteristic` і перевіряє критерій варіанта. `countQualifying` викликає `qualifies` для трьох чисел і повертає кількість успішних. `main` читає дані, викликає функцію підсумку та виводить результат.

Усі параметри передаються за значенням. Не використовуйте глобальні змінні, масиви, рядки, посилання, вказівники, рекурсію чи дублювання алгоритму аналізу цифр.

9. Індивідуальні варіанти

№	Характеристика та критерій	Input → Output
1	Сума всіх цифр; сума > 20	123456 555 10 → 1
2	Кількість парних цифр; не менше 3	123456 555 10 → 1
3	Кількість непарних цифр; не менше 3	123456 555 10 → 2
4	Найбільша цифра; дорівнює 9	123456 555 10 → 0
5	Найменша цифра; не більша за 2	123456 555 10 → 2
6	Кількість цифр > 5 ; не менше 2	123456 555 10 → 0
7	Кількість цифр < 5 ; не менше 2	123456 555 10 → 2
8	Сума парних цифр; > 10	123456 555 10 → 1
9	Сума непарних цифр; > 10	123456 555 10 → 1
10	Кількість нульових цифр; не менше 1	123456 555 10 → 1
11	Кількість цифр 5; не менше 2	123456 555 10 → 1
12	Сума цифр > 5 ; не менше 15	123456 555 10 → 0
13	Сума цифр < 5 ; не менше 6	123456 555 10 → 1
14	Кількість цифр, кратних 3; не менше 2	123456 555 10 → 1
15	Кількість цифр, не кратних 3; не менше 3	123456 555 10 → 2
16	Найбільша цифра мінус найменша; не менше 5	123456 555 10 → 1
17	Кількість цифр, більших за останню; не менше 2	123456 555 10 → 0
18	Кількість цифр, менших за останню; не менше 2	123456 555 10 → 1
19	Кількість цифр, що дорівнюють першій; не менше 2	123456 555 10 → 1
20	Кількість цифр, відмінних від першої; не менше 2	123456 555 10 → 1

Для 17–20 першу або останню цифру визначаєте арифметично, без рядків. Усі варіанти використовують той самий шаблон із трьох функцій.

10. Підготовка до захисту

Підготуйтеся пояснити виклик, параметр і аргумент, пройти функцію характеристики для нового числа, передбачити результат `qualifies`, змінити

поріг, пояснити, чому аргумент не змінюється під час передавання за значенням, перенести визначення після `main` і додати прототип.

11. Контрольні запитання

1. Що відбувається під час виклику функції?
2. Чим параметр відрізняється від аргументу?
3. Навіщо потрібен тип результату?
4. Що робить `return`?
5. Що відбувається з аргументом при передаванні за значенням?
6. Що таке локальна область видимості?
7. Навіщо потрібен прототип?
8. Що станеться при неправильній кількості аргументів?
9. Чому декомпозиція зменшує дублювання?
10. Як перевірити окрему функцію?

Лабораторна робота №6

Одновимірні масиви

1. Мета роботи

Навчитися зберігати набір однотипних значень в одному масиві, обходити його циклом і виконувати простий аналіз елементів.

Коли потрібно обробити десять однотипних значень, створювати десять окремих змінних незручно. Масив дозволяє зберегти набір під одним ім'ям і звертатися до кожного елемента за індексом. У цій роботі особливо важливий зв'язок між індексом і циклом: перший індекс дорівнює 0, а останній для масиву розміру `SIZE` — `SIZE - 1`. Це правило стане основою для наступної роботи з двовимірними масивами.

2. Очікувані результати

Після виконання роботи ви зможете:

1. Пояснити призначення масиву та оголосити масив фіксованого розміру;
2. Назвати допустимі індекси й звертатися до елемента через `array[index]`;
3. Заповнювати та виводити масив циклом;
4. Обчислювати суму, кількість і середнє вибраних елементів, знаходити перший і останній індекс та використовувати масив повторно;
5. Пояснити, чому цикл має умову `i < SIZE`, і не виходити за межі.

3. Що потрібно знати перед початком

Потрібні цикли, умови, лічильники та накопичувачі з попередніх лабораторних. Посилання, вказівники, динамічні масиви, `std::vector` і двовимірні масиви ще не використовуються.

4. Теоретичні відомості

Масив та індекс

Масив фіксованого розміру містить однотипні елементи під одним ім'ям:

У декларації `int values[SIZE]` квадратні дужки задають кількість елементів масиву. Після створення масиву запис `values[i]` використовує ті самі дужки вже для вибору одного елемента за індексом; це не означає, що індекс дорівнює розміру.

```
const int SIZE = 5;  
int values[SIZE];
```

Для `SIZE = 5` допустимі індекси `0, 1, 2, 3, 4`. `values[2]` означає третій елемент, а не число 2 як значення.

Обхід масиву

```
for (int i = 0; i < SIZE; ++i)
{
    std::cin >> values[i];
}
```

Умова `i < SIZE` гарантує, що останнім буде індекс `SIZE - 1`. Запис `i <= SIZE` додасть недопустимий індекс `SIZE` і може спричинити невизначену поведінку.

Сума, кількість і мінімум

```
int sum = 0;
int count = 0;
for (int i = 0; i < SIZE; ++i)
{
    sum += values[i];
    if (values[i] > 0)
    {
        ++count;
    }
}
```

Для непорожнього масиву мінімум починають із реального елемента:

```
int minValue = values[0];
int minIndex = 0;
for (int i = 1; i < SIZE; ++i)
{
    if (values[i] < minValue)
    {
        minValue = values[i];
        minIndex = i;
    }
}
```

`minValue` — значення, а `minIndex` — позиція цього значення. Якщо мінімум повторюється, строгий оператор `<` збереже перший індекс.

5. Приклади

Приклад 1. Введення та виведення

```
#include <iostream>

int main()
{
    const int SIZE = 5;
    int values[SIZE];
    for (int i = 0; i < SIZE; ++i)
    {
        std::cin >> values[i];
    }
    for (int i = 0; i < SIZE; ++i)
    {
        std::cout << i << ": " << values[i] << "\n";
    }
    return 0;
}
```

Для введення `4 8 -2 0 5` точний очікуваний вивід такий:

```
0: 4
1: 8
2: -2
3: 0
4: 5
```

Приклад 2. Сума та середнє

Після проходження масиву можна вивести `sum`, а дійсне середнє обчислити як `static_cast<double>(sum) / SIZE`. Перетворення потрібне до ділення.

6. Типові помилки

1. Перший елемент помилково шукають за індексом 1.
2. `i <= SIZE` виходить за межі.
3. `SIZE` плутають з останнім індексом.
4. Індекс `i` плутають зі значенням `values[i]`.
5. Елемент читають до того, як йому надали значення.
6. `sum` або `count` починають не з нуля.
7. Мінімум починають із 0, хоча всі елементи можуть бути додатними.
8. Використовують різні магічні межі замість одного `SIZE`.
9. Вважають, що доступ за межі безпечний, якщо програма один раз «працювала».
10. Створюють масив змінної довжини (VLA, variable-length array), наприклад `int values[sizeFromInput]`, у якого розмір визначається під час виконання; такий локальний масив не є стандартним C++.

Навмисно неправильний фрагмент:

```
for (int i = 0; i <= SIZE; ++i)
{
    std::cout << values[i] << " ";
}
```

Виправлення: замінити `i <= SIZE` на `i < SIZE`.

7. Міні-вправи

1. Назвіть допустимі індекси для `int values[7]`.
2. Для `int values[3] = {4, -2, 7}`; передбачте вивід циклу `for`, який послідовно друкує `values[i]` через пробіл. Очікуваний рядок: `4 -2 7`.
3. Знайдіть помилку в циклі для масиву розміру 10.
4. Запишіть індекс третього елемента.
5. Змініть суму всіх елементів на суму лише додатних.
6. Для `int values[3] = {4, -2, 7}`; простежте `i`, `values[i]` і `count` у циклі, де `count` збільшується лише за умови `values[i] > 0`. Заповніть таблицю після кожної ітерації; очікувані рядки: `0 | 4 | 1, 1 | -2 | 1, 2 | 7 | 2`.
7. Поясніть, чому мінімум починають із `values[0]`.

8. Лабораторне завдання

Оголосіть масив із 10 цілих чисел. Уведіть кожне значення в межах `[-1000, 1000]`, виведіть масив у форматі `індекс: значення`, а потім проаналізуйте його за умовою свого варіанта. Ця межа гарантує, що сума десяти елементів залишається в безпечному діапазоні для `int`.

Для вибраних елементів підрахуйте `count`, обчисліть `sum`, знайдіть `firstIndex` і `lastIndex`, а також виведіть їхні індекси у зростаючому порядку. Якщо `count > 0`, обчисліть `average = static_cast<double>(sum) / count`; інакше середнє не обчислюйте, а для обох індексів залиште значення `-1`.

Після цього виконайте другий прохід від останнього індексу до нульового. Виведіть вибрані індекси у зворотному порядку та замініть відповідні елементи масиву на `0`. Наприкінці ще раз виведіть масив, щоб було видно результат зміни.

Обов'язково використовуйте окремий цикл введення, окремий цикл виведення, цикл аналізу та зворотний прохід для зміни. Не використовуйте десять окремих змінних, `std::vector`, сортування, вказівники, посилання, двовимірні або динамічні масиви.

9. Індивідуальні варіанти

№	Умова
1	Додатні елементи: <code>value > 0</code> .
2	Від'ємні елементи: <code>value < 0</code> .
3	Нульові елементи: <code>value == 0</code> .
4	Парні елементи: <code>value % 2 == 0</code> .
5	Непарні елементи: <code>value % 2 != 0</code> .
6	Елементи, кратні 3: <code>value % 3 == 0</code> .
7	Елементи, не кратні 3: <code>value % 3 != 0</code> .
8	Елементи, кратні 5: <code>value % 5 == 0</code> .
9	Елементи, не кратні 5: <code>value % 5 != 0</code> .
10	Елементи, більші за 10: <code>value > 10</code> .
11	Елементи, менші за -10: <code>value < -10</code> .
12	Невід'ємні елементи: <code>value >= 0</code> .
13	Недодатні елементи: <code>value <= 0</code> .

№	Умова
14	Елементи в <code>[-5, 5]</code> : <code>value >= -5 && value <= 5</code> .
15	Елементи поза <code>[-5, 5]</code> : <code>value < -5 value > 5</code> .
16	Елементи в <code>[10, 20]</code> : <code>value >= 10 && value <= 20</code> .
17	Елементи в <code>[-20, -10]</code> : <code>value >= -20 && value <= -10</code> .
18	Додатні парні елементи: <code>value > 0 && value % 2 == 0</code> .
19	Додатні непарні елементи: <code>value > 0 && value % 2 != 0</code> .
20	Від'ємні парні елементи: <code>value < 0 && value % 2 == 0</code> .

10. Підготовка до захисту

Підготуйтеся назвати індекси для `SIZE = 10`, пояснити `i < SIZE`, показати значення `values[i]` на конкретній ітерації, змінити умову відбору, передбачити `count` і `sum`, а також знайти вихід за межі.

11. Контрольні запитання

1. Навіщо потрібен масив?
2. Що означає його розмір?
3. Чому перший індекс дорівнює 0?
4. Який останній індекс при `SIZE = 10`?
5. Що не так із `i <= SIZE`?
6. Чим індекс відрізняється від значення?
7. Як правильно ініціалізувати суму?
8. Як правильно ініціалізувати мінімум?
9. Що буде, якщо мінімум повторюється?
10. Чому доступ поза межами небезпечний?

Лабораторна робота №7

Двовимірні масиви

1. Мета роботи

Навчитися подавати табличні дані двовимірним масивом, звертатися до елемента за парою індексів і обробляти таблицю вкладеними циклами.

Одновимірний масив зручний, коли дані утворюють один ряд. Але оцінки студентів за кількома предметами, вимірювання за кілька днів або інша таблиця потребують двох координат: номера рядка й номера стовпця. Двовимірний масив зберігає таку структуру, а вкладені цикли проходять її рядок за рядом. У цій роботі прямокутні розміри 4×5 і 5×4 спеціально не робимо квадратними, щоб не плутати кількість рядків із кількістю стовпців.

2. Очікувані результати

Після виконання роботи ви зможете:

1. Оголосити прямокутний двовимірний масив і пояснити два індекси елемента;
2. Визначити межі рядків і стовпців;
3. Ввести та вивести таблицю вкладеними циклами;
4. Обчислити глобальну, рядкову або стовпцеву характеристику;
5. Пройти всі елементи без виходу за межі та не плутати індекси.

3. Що потрібно знати перед початком

Потрібні одновимірні масиви, індексація від нуля, умови та цикл `for`. `std::vector`, діапазонний `for`, вказівники, посилання й динамічна пам'ять ще не використовуються.

4. Теоретичні відомості

Два індекси

Для масиву 3×4 допустимі рядки `0, 1, 2` і стовпці `0, 1, 2, 3`. Елемент `matrix[1][2]` розташований у другому рядку та третьому стовпці.

```
const int ROWS = 3;
const int COLS = 4;
int matrix[ROWS][COLS];
```

Перший індекс відповідає рядку, другий — стовпцю. Допустимі межі: `0 <= row < ROWS` і `0 <= col < COLS`.

Вкладені цикли

Для таблиці 3×4 порядок обходу можна побачити так:

```
row = 0: col = 0, 1, 2, 3
row = 1: col = 0, 1, 2, 3
row = 2: col = 0, 1, 2, 3
```

```
for (int row = 0; row < ROWS; ++row)
{
    for (int col = 0; col < COLS; ++col)
    {
        // matrix[row][col]
    }
}
```

Зовнішній цикл вибирає рядок, а внутрішній проходить усі стовпці цього рядка. Для таблиці 2×3 порядок пар буде (0,0), (0,1), (0,2), (1,0), (1,1), (1,2).

Глобальна, рядкова та стовпцева характеристика

Для глобальної суми накопичувач створюють до обох циклів. Для суми рядка накопичувач обнуляють на початку кожного нового рядка. Для стовпцевої характеристики змінюють порядок циклів: зовнішнім стає `col`, внутрішнім — `row`.

5. Приклади

Приклад 1. Введення й виведення таблиці

```
#include <iostream>

int main()
{
    const int ROWS = 3;
    const int COLS = 4;
    int matrix[ROWS][COLS];
    for (int row = 0; row < ROWS; ++row)
    {
        for (int col = 0; col < COLS; ++col)
        {
            std::cin >> matrix[row][col];
        }
    }
    for (int row = 0; row < ROWS; ++row)
    {
        for (int col = 0; col < COLS; ++col)
        {
            std::cout << matrix[row][col] << " ";
        }
        std::cout << "\n";
    }
    return 0;
}
```

Для відтворення прикладу введіть матрицю пострічково:

```
1 2 3 4
5 6 7 8
-1 0 2 3
```

Точний вивід таблиці:

```
1 2 3 4
5 6 7 8
-1 0 2 3
```

Приклад 2. Сума рядків

```
for (int row = 0; row < ROWS; ++row)
{
    int rowSum = 0;
    for (int col = 0; col < COLS; ++col)
    {
        rowSum += matrix[row][col];
    }
    std::cout << rowSum << "\n";
}
```

`rowSum` обнуляється для кожного нового рядка, тому результати не змішуються.

Для цієї самої матриці очікувані суми рядків:

```
10
26
4
```

6. Типові помилки

1. `row <= ROWS` виходить за межі рядків.
2. Внутрішній цикл використовує `ROWS` замість `COLS`.
3. `matrix[col][row]` міняє місцями координати.
4. Накопичувач рядка не обнуляють на початку нового рядка.
5. Приклади роблять лише квадратними й перестають помічати різницю розмірів.
6. Використовують масиви змінної довжини (VLA), у яких розмір визначається під час виконання; це не є стандартним C++.
7. Застосовують `std::vector` або динамічну пам'ять до вивчення цих тем.

7. Міні-вправи

1. Назвіть індекси для `int a[4][6]`.
2. Який елемент позначає `a[2][3]`?
3. Для масиву `int a[4][5]` скільки ітерацій внутрішнього циклу буде для **одного рядка**? Відповідь: **5**.
4. Для того самого масиву скільки разів **загалом виконається тіло внутрішнього циклу** під час повного обходу? Відповідь: **20**.
5. Знайдіть помилку в `row <= ROWS`.
6. Де обнулити `rowSum`?
7. Поясніть різницю між обходом рядків і стовпців.
8. Назвіть послідовність пар для масиву **2 × 3**.

8. Лабораторне завдання

Працюйте з прямокутним масивом розміру 4×5 або 5×4 відповідно до варіанта. Уводьте кожен елемент у межах $[-1000, 1000]$. Введіть усі елементи, виведіть таблицю, обчисліть одну глобальну характеристику та одну характеристику для кожного рядка або стовпця. За такої межі суми щонайбільше 20 елементів залишаються в безпечному діапазоні для `int`.

Обов'язково використовуйте вкладені `for` та `if`, іменовані константи `ROWS` і `COLS`, а також коректні межі індексів. Не використовуйте `std::vector`, `new`, `delete`, вказівники, STL-алгоритми, сортування або масиви змінної довжини (VLA).

9. Індивідуальні варіанти

№	Розмір	Глобальна характеристика	Характеристика рядків/стовпців
1	4×5	Сума додатних	Кількість від'ємних у кожному рядку
2	5×4	Кількість парних	Сума непарних у кожному стовпці
3	4×5	Сума елементів < 10	Кількість елементів ≥ 10 у кожному рядку
4	5×4	Кількість нулів	Сума ненульових у кожному стовпці
5	4×5	Сума парних	Кількість непарних у кожному рядку
6	5×4	Кількість додатних	Сума від'ємних у кожному стовпці
7	4×5	Сума елементів у $[-10, 10]$	Кількість елементів поза діапазоном у кожному рядку
8	5×4	Кількість елементів > 20	Сума елементів ≤ 20 у кожному стовпці
9	4×5	Сума від'ємних	Кількість невід'ємних у кожному рядку
10	5×4	Кількість кратних 3	Сума некрatних 3 у кожному стовпці
11	4×5	Сума елементів > 5	Кількість елементів ≤ 5 у кожному рядку
12	5×4	Кількість елементів < -5	Сума елементів ≥ -5 у кожному стовпці
13	4×5	Сума кратних 5	Кількість некрatних 5 у кожному рядку
14	5×4	Кількість елементів у $[1, 20]$	Сума елементів поза $[1, 20]$ у кожному стовпці

№	Розмір	Глобальна характеристика	Характеристика рядків/стовпців
15	4×5	Сума ненульових	Кількість нулів у кожному рядку
16	5×4	Кількість кратних 4	Сума некратних 4 у кожному стовпці
17	4×5	Сума елементів < 0	Кількість елементів > 0 у кожному рядку; нуль не входить
18	5×4	Кількість елементів < -10	Сума елементів ≥ -10 у кожному стовпці
19	4×5	Сума елементів у $[-15, 15]$	Кількість елементів поза діапазоном у кожному рядку
20	5×4	Кількість елементів у $[-20, 20]$	Сума елементів поза діапазоном у кожному стовпці

Для варіанта 19 перевіряйте діапазон через `value >= -15 && value <= 15`, без `std::abs` як нової теми.

Приклад оформлення

Для варіанта 1 введіть 20 чисел у масив 4×5 , виведіть таблицю, знайдіть глобальну суму додатних і чотири кількості від'ємних — по одній для кожного рядка. Умова має містити `input`, обмеження, `output` і обов'язкове використання вкладених `for`.

10. Підготовка до захисту

Підготуйтеся пояснити межі, пройти дві-три ітерації вкладених циклів, змінити умову, поміняти 4×5 на 5×4 , знайти помилку індексу та пояснити, де створюється накопичувач.

11. Контрольні запитання

1. Які індекси має `int a[3][5]`?
2. Скільки разів виконається внутрішній цикл для 4×5 ?
3. Чому `row <= ROWS` є помилкою?
4. Чим відрізняються `a[row][col]` і `a[col][row]`?
5. Де ініціалізувати суму рядка?
6. Як змінити порядок циклів для стовпців?
7. Чому прямокутний масив корисний для пояснення індексів?
8. Чому масиви змінної довжини (VLA) не використовують у стандартному C++?
9. Чим глобальна сума відрізняється від суми рядка?
10. Що станеться, якщо переплутати `ROWS` і `COLS`?

Лабораторна робота №8

Символи та рядки

1. Мета роботи

Навчитися зберігати текст у `std::string`, читати рядок із пробілами, проходити його елементи `char` за індексом і виконувати просту обробку.

Окремий `char` зручно використовувати для одного символу, але повідомлення й речення складаються з багатьох символів. `std::string` дає змогу зберегти весь текст і звертатися до його елементів за індексом. Функція-предикат — функція, що повертає `bool` і перевіряє певну властивість значення. У цій роботі ми навчимося читати рядок із пробілами, проходити його символ за символом і будувати такі функції для підрахунку. Основне завдання обмежене ASCII-наборами, щоб зосередитися на індексації та умовах, а не на складному опрацюванні Unicode.

2. Очікувані результати

Після виконання роботи ви зможете:

1. Пояснити різницю між `char` і `std::string`, а також розрізнити `'A'` і `"A"`;
2. Прочитати рядок із пробілами через `std::getline`;
3. Отримати довжину та пройти рядок за індексом;
4. Підрахувати символи за умовою і змінити символи в копії рядка;
5. Обробити порожній рядок та пояснити обмеження ASCII.

3. Що потрібно знати перед початком

Потрібні змінні, умови, цикли, функції та одновимірні масиви. Посилання, вказівники, C-рядки, регулярні вирази та складне опрацювання Unicode ще не використовуються.

4. Теоретичні відомості

`char` і `std::string`

Один символ записують у одинарних лапках: `'A'` має тип `char`. Текстовий літерал записують у подвійних лапках: `"A"` є рядком, навіть якщо містить лише один видимий символ.

```
char symbol = 'A';  
std::string text = "A";
```

Рядок можна уявляти як послідовність символів. Якщо `text.size()` дорівнює `n`, логічні символи рядка мають позиції від `0` до `n - 1`. Тому звичайний цикл обробки символів використовує умову `i < text.size()`.

Технічна примітка: стандартний рядок має службову завершальну позицію `text[text.size()]` зі значенням `\0`. Вона не належить до логічного вмісту рядка. Для порожнього рядка `text.size()` дорівнює нулю, тому циклу з умовою `i < text.size()` просто нема що обробляти.

`std::cin` і `std::getline`

`std::cin >> text` читає до першого пробільного символу. `std::getline(std::cin, text)` читає весь рядок разом із внутрішніми пробілами. Якщо перед `getline` було `std::cin >> value`, у потоці може залишитися символ нового рядка `'\n'` або інші пробільні символи:

`std::cin.ignore()` без аргументів відкидає лише один символ, тому така форма не є універсальним способом відновлення потоку після довільного некоректного введення. Щоб перейти від форматованого введення до всього рядка, відкиньте символи до кінця поточного рядка:

```
#include <limits>

std::cin >> value;
std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
std::getline(std::cin, text);
```

Крапка в `text.length()` пов'язує змінну `text` з операцією `length`; у цьому практикумі достатньо сприймати її як спосіб звернутися до операції конкретного значення.

У цій роботі очікується коректне введення; наведений `ignore` очищає залишок поточного рядка, але не призначений для відновлення потоку після довільного некоректного введення.

Індексний прохід

```
int length = static_cast<int>(text.length());
for (int i = 0; i < length; ++i)
{
    std::cout << text[i] << '\n';
}
```

Умова `i < length` обробляє лише логічні символи. Межа `i <= length` додала б позицію завершального `\0`, яка не є символом логічного вмісту. Для порожнього рядка `length` дорівнює нулю, тому цикл не виконається.

ASCII-обмеження

```
bool isUppercaseLatin(char c)
{
    return c >= 'A' && c <= 'Z';
}
```

Такий предикат призначений для латинських ASCII-літер. Не робіть висновок, що кожен `char` є однією видимою Unicode-літерою: у UTF-8 одна літера може складатися з кількох елементів `char`.

5. Приклади

Приклад 1. Рядок із пробілами

```
#include <iostream>
#include <string>

int main()
{
    std::string text;
    std::getline(std::cin, text);
    std::cout << text << '\n';
    return 0;
}
```

Введення `Hello world` читається повністю, на відміну від `std::cin >> text`.

Приклад 2. Підрахунок великих латинських літер

```
int count = 0;
for (int i = 0; i < static_cast<int>(text.length()); ++i)
{
    if (isUppercaseLatin(text[i]))
    {
        ++count;
    }
}
```

Використайте предикат `isUppercaseLatin`, визначений у попередньому підрозділі, для кожного `text[i]` і збільшуйте лічильник лише для істинного результату. Для `text = "Room 204"` є одна велика латинська літера (**R**), тому очікуваний результат підрахунку — `1`.

Приклад 3. Модифікація копії

```
std::string modified = text;
for (int i = 0; i < static_cast<int>(modified.length()); ++i)
{
    if (modified[i] >= '0' && modified[i] <= '9')
    {
        modified[i] = '#';
    }
}
```

Для `Room 204` оригінал залишиться без змін, а копія стане `Room ###`.

6. Типові помилки

1. `char c = "A"` використовує рядковий, а не символьний літерал.

2. `std::cin >> text` обрізає введення на першому пробілі.
3. `getline` після `std::cin >>` може одразу прочитати символ нового рядка `'\n'`, що залишився у вхідному потоці.
4. `i <= length` помилково включає позицію завершального `\0` до звичайної обробки; для логічного вмісту потрібна умова `i < length`.
5. У порожньому рядку `text[0]` не є першим логічним символом: це спеціальна позиція завершального `\0`, яку не можна трактувати як вміст або змінювати.
6. `'5'` і `5` мають різні типи й значення.
7. ASCII-предикат не є універсальним Unicode-алгоритмом.

7. Міні-вправи

1. Передбачте результат проходу рядка `ABC`.
2. Поясніть, чому цикл обробки рядка має `i < text.size()`, а не `i <= text.size()`.
3. Поясніть різницю між `'7'` і `7`.
4. Чому `getline` може прочитати порожній рядок після `cin >>`?
5. Скільки ітерацій має цикл для порожнього рядка?
6. Допишіть умову заміни ASCII-цифри на `#`.

8. Лабораторне завдання

Прочитайте один ASCII-рядок довжиною до 200 символів через `std::getline`. Для свого варіанта визначте дві групи символів А і В.

Програма має порахувати кількість символів групи А, порахувати кількість символів групи В, створити копію рядка та замінити в копії лише символи групи А на `#`.

Використайте дві окремі функції-предикати з параметром `char` за значенням і поверненням `bool`. Рядок проходите індексним `for`. Не використовуйте `find`, `replace`, регулярні вирази, C-рядки, вказівники, посилання чи STL-алгоритми.

9. Індивідуальні варіанти

№	Група А	Група В
1	A B C D E	F G H I J
2	a b c d e	f g h i j
3	K L M N O	P Q R S T
4	k l m n o	p q r s t
5	U V W X Y	A E I O U
6	u v w x y	a e i o u
7	A C E G I	B D F H J

№	Група А	Група В
8	k l m n o	a c e g i
9	1 2 3 4 5	6 7 8 9 0
10	! ? . , :	; - _ + =
11	A D G J M	N Q T W Z
12	a d g j m	n q t w z
13	B D F H J	L N P R T
14	b d f h j	l n p r t
15	C F I L O	R U X Y Z
16	c f i l o	r u x y z
17	0 1 2 3 4	5 6 7 8 9
18	@ # \$ % &	* + - = _
19	A F K P U	E J O T Y
20	a f k p u	e j o t y

Для кожного варіанта використайте ASCII input із символом групи А, символом групи В, символом поза обома групами та, за можливості, пробілом. Наприклад для варіанта 1: A F x B J 7 → Count A: 2, Count B: 2, modified # F x # J 7.

10. Підготовка до захисту

Підготуйтеся пояснити `getline`, пройти кілька ітерацій, пояснити `i < length`, змінити символ групи А, розрізнити '5' і 5, обробити порожній рядок і пояснити ASCII-обмеження.

11. Контрольні запитання

1. Чим `char` відрізняється від `std::string`?
2. Чим `'A'` відрізняється від `"A"`?
3. Чим `std::cin >> text` відрізняється від `std::getline`?
4. Що означає символ нового рядка `'\n'`, який може залишитися у вхідному потоці після `std::cin >>?`
5. Що повертає `length()`?
6. Які позиції належать до логічного вмісту рядка і що означає `text[text.size()]`?
7. Чому звичайний цикл обробки використовує `i < text.size()`, навіть якщо `text[text.size()]` має спеціальне значення?
8. Що робить цикл для порожнього рядка?

Лабораторна робота №9

Посилання, вказівники та базова динамічна пам'ять

1. Мета роботи

Сформуванати цілісну модель: значення → посилання → адреса → значення за адресою, а також безпечно створити й звільнити один динамічний об'єкт.

До цього моменту ви передавали дані у функції як значення. У такому разі функція отримує копію і не змінює початкову змінну. У цій роботі послідовно розрізняємо значення, інше ім'я того самого об'єкта, адресу об'єкта та доступ до значення за адресою. Посилання дає змогу працювати з тим самим об'єктом, а вказівник зберігає адресу. Наприкінці використаємо цю модель для одного динамічно створеного об'єкта.

2. Очікувані результати

Після виконання роботи ви зможете:

1. Пояснити різницю між значенням, посиланням і вказівником;
2. Передавати змінну за посиланням і використовувати `const int&` або `const double&` для читання;
3. Отримувати адресу через `&`, зберігати її у вказівнику та читати значення через `*`;
4. Перевіряти `nullptr` до розіменування й не змінювати дані для порожнього вказівника;
5. Створити один об'єкт через `new`, звільнити його через `delete` і не використовувати адресу після цього.

У цьому розділі шаблони не використовуються. У власному коді всюди пишіть конкретний тип свого варіанта — `int` або `double` — у параметрах `const int&` / `const double&` та `int*` / `double*`.

3. Що потрібно знати перед початком

Потрібні змінні, функції, параметри за значенням, умови, цикли, масиви та рядки. Масиви вказівників, `new[]`, арифметика вказівників, вказівник на вказівник, розумні вказівники й класи не входять до цієї роботи.

4. Теоретичні відомості

Робоче визначення об'єкта

У цій лабораторній об'єктом називатимемо конкретне значення, яке існує під час виконання програми і до якого можна звертатися через змінну, посилання

або адресу. Наприклад, `int value = 10;` створює один об'єкт цілого типу зі значенням 10.

Проблема параметра за значенням

Параметр за значенням отримує окрему копію аргументу. Зміна цієї копії закінчується разом із виконанням функції:

```
void addOne(int value)
{
    ++value;
}
```

Якщо викликати `addOne(number)`, зовнішня змінна `number` не зміниться. Функція працює з локальним параметром `value`, який спочатку має те саме значення.

Посилання як інше ім'я об'єкта

Посилання — це інше ім'я вже існуючого об'єкта. У декларації `int& ref = value` символ `&` означає посилання. Посилання не створює копії та має бути ініціалізоване одразу.

```
int value = 10;
int& ref = value;
ref = 25;
```

Після присвоєння через `ref` значення `value` дорівнює 25, тому що обидва імені звертаються до того самого об'єкта.

Параметр функції за посиланням

Якщо функція має параметр `int&`, вона може змінювати початковий об'єкт:

```
void addOne(int& value)
{
    ++value;
}
```

Виклик `addOne(number)` змінює `number`. Це відрізняється від параметра `int value`, який працює з копією.

const T& для читання

Параметр `const int&` або `const double&` дає функції доступ до вже існуючого об'єкта без права змінювати його через цей параметр. На базовому рівні це означає: функція читає передане значення і не створює окремої копії.

```
int twice(const int& value)
{
    return value * 2;
}
```

Адреса об'єкта

Адреса — це дані про місце, за яким під час виконання можна знайти об'єкт. У виразі `&value` символ `&` отримує адресу об'єкта `value`. Це інша роль, ніж `&` у декларації `int& ref`.

Вказівник і розіменування

Вказівник — це змінна, яка зберігає адресу об'єкта. У декларації `int* ptr` символ `*` після типу означає вказівник, а у виразі `*ptr` — доступ до об'єкта за збереженою адресою:

```
int value = 10;
int* ptr = &value;
```

Тут `value` — об'єкт зі значенням 10, `&value` — його адреса, `ptr` — змінна з цією адресою, а `*ptr` — значення об'єкта. Через `*ptr = 25` можна змінити `value`.

nullptr і попередня перевірка

`nullptr` — спеціальне значення вказівника, яке означає «вказівник не зберігає адресу об'єкта». Такий вказівник не можна розіменовувати. Перед доступом через `*ptr` перевірте, що `ptr` не дорівнює `nullptr`:

```
if (ptr != nullptr)
{
    std::cout << *ptr << '\n';
}
```

Функція з параметром-вказівником має спочатку виконати перевірку, а вже потім розіменовувати вказівник:

```
bool increase(int* ptr)
{
    if (ptr == nullptr)
    {
        return false;
    }
    ++(*ptr);
    return true;
}
```

Для `nullptr` ця функція повертає `false` і не змінює жодного об'єкта. Для дійсної адреси вона змінює значення та повертає `true`.

Коротке порівняння посилання і вказівника

Властивість	Посилання	Вказівник
Що представляє	інше ім'я вже існуючого об'єкта	змінну, що зберігає адресу
Ініціалізація	потрібна під час створення	може містити <code>nullptr</code>
Доступ до значення	через ім'я посилання	через розіменування <code>*</code> після перевірки

Властивість	Посилання	Вказівник
Зміна зв'язку	після ініціалізації не переприв'язується	може зберігати іншу адресу
Використання тут	змінити аргумент у функції	перевірити адресу, змінити об'єкт і показати <code>nullptr</code>

Один динамічний об'єкт

`new int` створює один динамічний об'єкт і повертає його адресу. Цю адресу зберігає вказівник. Через `*number` можна звернутися до значення об'єкта. `delete number` завершує час життя цього об'єкта. Після цього старий вказівник не можна розіменувати, тому його встановлюють у `nullptr`:

```
int* number = new int;
*number = 42;
std::cout << *number << '\n';
delete number;
number = nullptr;
```

`number = nullptr` саме по собі не звільняє пам'ять. Пам'ять звільняє `delete`; присвоєння `nullptr` після цього лише прибирає стару адресу зі змінної-вказівника. Забутий `delete` створює витік пам'яті.

`new` і `delete` тут використовуються лише для навчального пояснення адреси та часу життя одного об'єкта. У прикладному сучасному C++ зазвичай застосовують засоби автоматичного керування ресурсами; їх у цій роботі не вводимо.

5. Приклади

Приклад 1. Передавання за значенням і посиланням

```
#include <iostream>

void increaseByValue(int value)
{
    ++value;
}

void increaseByReference(int& value)
{
    ++value;
}

int main()
{
    int number = 10;

    increaseByValue(number);
    std::cout << number << '\n';

    increaseByReference(number);
    std::cout << number << '\n';

    return 0;
}
```

Перший вивід — 10: під час `increaseByValue(number)` створюється локальна копія, яка зникає після повернення з функції. Другий вивід — 11: під час `increaseByReference(number)` параметр пов'язаний із тим самим об'єктом.

Порядок виконання:

1. `number` створюється зі значенням 10;
2. Функція з параметром за значенням збільшує лише локальну копію;
3. Функція з параметром-посиланням збільшує початковий об'єкт;
4. Після кожного виклику перевіряйте значення зовнішньої змінної.

Приклад 2. Обмін двох значень через посилання

```
#include <iostream>

void swapValues(int& a, int& b)
{
    int temporary = a;
    a = b;
    b = temporary;
}

int main()
{
    int first = 3;
    int second = 8;

    swapValues(first, second);
    std::cout << first << ' ' << second << '\n';

    return 0;
}
```

Програма виводить 8 3. `std::swap` тут навмисно не використовується, щоб було видно навчальну дію обміну.

Приклад 3. Читання через const-посилання

```
#include <iostream>

int twice(const int& value)
{
    return value * 2;
}

int main()
{
    int number = 7;
    std::cout << twice(number) << '\n';
    return 0;
}
```

Програма виводить 14. Функція читає `number`, але не може змінити його через параметр `const int&`.

Приклад 4. Адреса, вказівник і розіменування

```
#include <iostream>

int main()
{
    int value = 10;
    int* ptr = &value;

    std::cout << value << '\n';
    std::cout << &value << '\n';
    std::cout << ptr << '\n';
    std::cout << *ptr << '\n';

    *ptr = 25;
    std::cout << value << '\n';

    return 0;
}
```

Адреса у двох запусках може бути різною, тому її конкретні цифри не є частиною очікуваного результату. Важливі співвідношення: `ptr` і `&value` позначають одну адресу, `*ptr` спочатку дорівнює 10, а після зміни `value` дорівнює 25.

Приклад 5. Безпечна перевірка nullptr

```
#include <iostream>

bool increase(int* ptr)
{
    if (ptr == nullptr)
    {
        return false;
    }
    ++(*ptr);
    return true;
}

int main()
{
    int value = 10;
    int* valid = &value;
    int* empty = nullptr;

    std::cout << increase(valid) << ' ' << value << '\n';
    std::cout << increase(empty) << ' ' << value << '\n';

    return 0;
}
```

Очікувані ознаки результату — 1 11 для дійсної адреси і 0 11 для nullptr. Перевірка відбувається до *ptr, тому порожній вказівник не розіменовується.

Приклад 6. Один динамічний int

```
#include <iostream>

int main()
{
    int* number = new int;
    *number = 42;
    std::cout << *number << '\n';

    delete number;
    number = nullptr;

    return 0;
}
```

У прикладі є рівно один new і рівно один відповідний delete. Після delete програма не читає *number; після присвоєння nullptr вказівник уже не зберігає адресу звільненого об'єкта.

Приклад 7. Функція, яка змінює значення через вказівник

```
#include <iostream>

bool setToZero(int* value)
{
    if (value == nullptr)
    {
        return false;
    }
    *value = 0;
    return true;
}

int main()
{
    int number = 5;
    int* valid = &number;
    int* empty = nullptr;

    std::cout << setToZero(valid) << ' ' << number << '\n';
    std::cout << setToZero(empty) << ' ' << number << '\n';

    return 0;
}
```

Функція повертає `true` і змінює число для дійсної адреси. Для `nullptr` вона повертає `false` і завершується до розіменування.

6. Типові помилки

Посилання помилково вважають копією

`int& ref = value` не створює другого об'єкта зі скопійованим значенням. Запис `ref = 25` змінює `value`, бо `ref` є іншим ім'ям того самого об'єкта.

Плутанина ролей &

У `int& ref = value` символ `&` оголошує посилання. У `int* ptr = &value` вираз `&value` отримує адресу. Це різні ролі одного символу.

Плутанина ролей *

У `int* ptr` символ `*` оголошує тип вказівника. У `*ptr = 10` він означає доступ до об'єкта за адресою. Друга форма допустима лише тоді, коли `ptr` містить дійсну адресу.

Неініціалізований вказівник

Навмисно неправильний фрагмент. Не запускати як коректний приклад.

```
int* ptr;
*ptr = 10;
```

Вказівник не отримав адресу об'єкта, тому його розіменування не має безпечного результату. Ініціалізуйте вказівник адресою існуючого об'єкта або значенням `nullptr`.

Розіменування nullptr

Навмисно неправильний фрагмент. Не запускати як коректний приклад.

```
int* ptr = nullptr;
std::cout << *ptr << '\n';
```

Перед розіменуванням потрібна перевірка `ptr != nullptr`. Не використовуйте навмисне розіменування `nullptr` у робочій програмі.

delete для звичайного об'єкта

Навмисно неправильний фрагмент. Не запускати як коректний приклад.

```
int value = 10;
delete &value;
```

`value` не створено через `new`, тому його адресу не можна передавати до `delete`. Звичайний локальний об'єкт завершує час життя автоматично.

Повторний delete

Навмисно неправильний фрагмент. Не запускати як коректний приклад.

```
int* ptr = new int;
delete ptr;
delete ptr;
```

Після першого `delete` динамічний об'єкт уже звільнено. Повторне звільнення тієї самої адреси є помилкою.

Використання вказівника після delete

Навмисно неправильний фрагмент. Не запускати як коректний приклад.

```
int* ptr = new int;
*ptr = 42;
delete ptr;
std::cout << *ptr << '\n';
```

Після `delete` об'єкта вже немає, тому старий вказівник не можна розіменовувати. Після звільнення встановлюйте його у `nullptr` і не використовуйте для читання.

ptr = nullptr без delete

Навмисно неправильний порядок дій. Не використовувати як коректний спосіб звільнення.

```
int* ptr = new int;
ptr = nullptr;
```

Так втрачається адреса динамічного об'єкта, але сам об'єкт не звільняється. Правильний порядок: `delete ptr; ptr = nullptr;`.

Хибне твердження «масив — це вказівник»

Масив і вказівник — різні типи. У багатьох виразах ім'я масиву перетворюється на адресу його першого елемента, але це не робить масив вказівником.

7. Міні-вправи

Зразки схем пам'яті

Схема має показувати не фізичну будову пам'яті, а зв'язок між іменем, адресою, вказівником і значенням. Художнє оформлення не оцінюється.

Для локальної змінної та вказівника достатньо такої нотації:

```
локальний об'єкт:  
[ value: 10 ]  
адреса A  
  
ptr:  
[ зберігає адресу A ]  
|  
+---- зберігає адресу ----> [ value: 10 ]  
  
після *ptr = 25:  
[ зберігає адресу A ]  
|  
+---- зберігає адресу ----> [ value: 25 ]
```

Для одного динамічного об'єкта покажіть окремо змінну-вказівник і об'єкт:

```
до delete:  
dynamicPtr:  
[ зберігає адресу B ]  
|  
+---- зберігає адресу ----> [ динамічний об'єкт: 42 ]  
  
після delete dynamicPtr:  
[ стара адреса більше не веде до об'єкта ]  
  
після dynamicPtr = nullptr:  
[ dynamicPtr = nullptr ]  
без стрілки до об'єкта
```

У кожній схемі обов'язково позначте окремий блок для вказівника, окремий блок для об'єкта або його значення, стрілку з підписом «зберігає адресу», значення до і після зміни, а для динамічного випадку — стан після `delete` і після присвоєння `nullptr`.

1. Передбачте значення після функції з параметром за значенням.
2. Передбачте значення після функції з `int&`.
3. Що зберігає `int* ptr`?
4. Що виведе `*ptr`, якщо `ptr = &value`?
5. Знайдіть помилку в розіменуванні `nullptr`.
6. Чому після `delete ptr` не можна читати `*ptr`?
7. Чи звільняє пам'ять `ptr = nullptr` без `delete`?
8. Намалюйте схему для `int value; int* ptr = &value;`.

9. Доповніть функцію з `const`-посиланням і визначте результат виклику:

```
int twice(const int& value)
{
    return value * 2;
}
```

Для `int number = 7` очікуваний результат `twice(number)` — 14. Поясніть, чому функція не може змінити `value` через цей параметр.

10. Вставте пропущені дії у правильному порядку:

```
int* ptr = nullptr;
ptr = /* 1 */;
/* 2 */
std::cout << *ptr << "\n";
/* 3 */
/* 4 */
```

Використайте рівно такі дії: `new int`, `*ptr = 42`, `delete ptr`, `ptr = nullptr`. Очікуваний вивід — 42, а кінцевий стан вказівника — `nullptr`.

8. Лабораторне завдання

У всіх варіантах використовуйте три введені значення конкретного типу `int` або `double`, зазначеного в таблиці. Для `int` уведіть значення в межах `[-100, 100]`, для `double` — у межах `[-100.0, 100.0]`. Ці межі є частиною умови та достатні для наведених у таблиці операцій.

Обмеження роботи

- Використовуйте `std::` і не пишіть глобальний `using namespace std;`.
- Для трьох операцій створіть окремі функції: з параметром-посиланням, із `const`-посиланням і з параметром-вказівником.
- Перевіряйте вказівник до розіменування. Виклик із `nullptr` не повинен змінювати дані.
- Створіть рівно один числовий об'єкт через `new`, виконайте один відповідний `delete`, а потім запишіть у вказівник `nullptr`.
- Не використовуйте `std::swap`, `new[]`, `delete[]`, арифметику вказівників, розумні вказівники, масиви або конструкції, яких ще не було в курсі.

Етап А. Значення, посилання та вказівник

Виконуйте дії саме в такому порядку:

1. Зчитайте три значення та виконайте операцію через посилання над першою змінною; якщо в таблиці наведено обмін, обміняйте вказані змінні через посилання;
2. Викличте функцію з `const`-посиланням для значення з таблиці. Вона лише читає дані й повертає свій результат;
3. Створіть вказівник правильного типу та викличте функцію з адресою **першої введеної змінної** — `&a`, `&x`, `&p` або відповідної першої змінної

вашого варіанта. Для варіантів 5 і 15 передайте `y` окремим параметром за значенням;

4. Викличте цю функцію ще раз із `nullptr`. Вона має спочатку перевірити вказівник, повернути `false` і не змінити дані;
5. Виведіть результати обох викликів і поточне значення першої змінної.

Контрольна точка між етапами

Перед переходом до Етапу В запишіть поточні значення всіх трьох змінних після операції через посилання та дійсної операції через вказівник. Перевірте, що виклик із `nullptr` повернув `false` і не змінив першу змінну. Коротко простежте цей стан у звіті.

Етап В. Один динамічний об'єкт

Виконуйте Етап В лише після завершення Етапу А:

1. Створіть рівно один динамічний об'єкт потрібного типу через `new` і запишіть у нього значення за формулою таблиці, використовуючи **поточні** значення після Етапу А;
2. Змініть динамічне значення за другою формулою, виведіть його, виконайте один `delete` і одразу встановіть цей вказівник у `nullptr`. Після `delete` не розіменовуйте його.

9. Індивідуальні варіанти

У таблиці один рядок описує обидва етапи роботи. У стовпці «Етап А» спочатку виконано операцію через посилання, потім — читання через `const`-посилання і зміну першої змінної через вказівник. Функцію з параметром-вказівником викликайте з адресою **першої змінної**, а потім із `nullptr`.

№	Тип і вхід	Етап А: посилання, <code>const&</code> , вказівник	Етап В: один <code>new</code> і зміна
1	<code>int a,b,c</code>	<code>a += b; const&: c*2; ptr: a+1</code>	<code>new: a+b+c</code> ; потім <code>-2</code>
2	<code>double x,y,z</code>	<code>x -= y; const&: z/2.0; ptr: x*2</code>	<code>new: x+y+z</code> ; потім <code>+1.0</code>
3	<code>int p,q,r</code>	обмін <code>p,q</code> ; <code>const&: r%10</code> ; <code>ptr: p*2</code>	<code>new: p+q+r</code> ; потім <code>-1</code>
4	<code>double a,b,c</code>	<code>a += b; const&: c+1.0; ptr: -a</code>	<code>new: a*b+c</code> ; потім <code>+2.0</code>

№	Тип і вхід	Етап А: посилання, const&, вказівник	Етап В: один new і зміна
5	int x,y,z	x -= z; const&: y*y; ptr: x+y	new: x+z; потім *2
6	double p,q,r	p = p+q-r; const&: q/2.0; ptr: p-0.5	new: p+q+r; потім *2.0
7	int a,b,c	a += 2*b; const&: c%2; ptr: a-3	new: a+b+c; потім +5
8	double x,y,z	x *= y; const&: z+2.0; ptr: x/2.0	new: x+y-z; потім +3.0
9	int m,n,k	m -= n; const&: k*3; ptr: -m	new: m+n+k; потім +4
10	double a,b,c	a += b/2.0; const&: c*2.0; ptr: a+1.5	new: a+b+c; потім змінити знак
11	int a,b,c	a = a+b-c; const&: b%2; ptr: a+2	new: a+b+c; потім -5
12	double x,y,z	x -= z; const&: y/3.0; ptr: x*3.0	new: x+y+z; потім +2.0
13	int a,b,c	a = a+b-c; const&: модуль c через if; ptr: a*2	new: a+b+c; потім -1
14	double a,b,c	a += b; const&: c/3.0; ptr: -a	new: a*b+c; потім +2.0
15	int x,y,z	x -= z; const&: 2*y; ptr: x+y	new: x+z; потім *3
16	double p,q,r	p = p+q-r; const&: r/2.0; ptr: p+0.25	new: p+q+r; потім *2.0
17	int a,b,c	a = a+2*b-c; const&: 1 для c<0; ptr: a-4	new: a+b+c; потім +7

№	Тип і вхід	Етап А: посилання, const&, вказівник	Етап В: один new і зміна
18	double x,y,z	x *= y; const&: z+1.0; ptr: x/2.0	new: x+y-z; потім +5.0
19	int m,n,k	m += n; const&: k*k; ptr: -m	new: m+n+k; потім -3
20	double a,b,c	a = a+b/2.0; const&: 2*c; ptr: a+2.5	new: a+b+c; потім змінити знак

Для кожного варіанта виведіть проміжні результати з підписами. Для дійсної адреси покажіть `true` і змінене значення, для `nullptr` — `false` та відсутність зміни. У варіантах 5 і 15 передайте `y` до функції з параметром-вказівником окремим параметром за значенням.

10. Підготовка до захисту

Підготуйтеся:

1. Замінити параметр-посилання `int&` на параметр за значенням `int` і спрогнозувати зміну результату;
2. Пояснити кожен символ у `int* ptr = &x;`
3. Змінити `*ptr` і назвати змінну, яка зміниться;
4. Написати функцію з параметром-вказівником, що повертає `false` для `nullptr`;
5. Пояснити, чому після `delete` не можна читати `*ptr`;
6. Пояснити різницю між `delete ptr` і `ptr = nullptr`;
7. Виконати коротке покрокове трасування конкретних чисел: значення до і після операції через посилання, результат функції читання, значення після дійсного виклику через вказівник, результат виклику з `nullptr`, значення динамічного об'єкта та його стан після `delete`.

11. Контрольні запитання

1. Чим параметр за значенням відрізняється від посилання?
2. Що означає `int& ref = value`?
3. Чи створює посилання копію?
4. Яка різниця між `&` у `int& ref` і `&value`?
5. Що зберігає вказівник?
6. Що означає `*ptr` у виразі?
7. Яка різниця між `int* ptr` і `*ptr = 5`?
8. Для чого використовується `nullptr`?

9. Чому не можна розіменувати `nullptr`?
10. Що станеться, якщо функція з параметром-вказівником перевіряє `nullptr` після `*ptr`?
11. Що робить `new int`?
12. Що робить `delete`?
13. Чому після `delete` старий вказівник не можна розіменувати?
14. Чи звільняє пам'ять просте `ptr = nullptr` без `delete`?
15. Чому твердження «масив — це вказівник» технічно неправильне?

Лабораторна робота №10

Текстові файли

1. Мета роботи

Навчитися читати текстові дані з файла та записувати результат у текстовий файл через файлові потоки.

Значення у змінних існують лише під час роботи програми. Якщо потрібно зберегти дані або опрацювати їх пізніше, потрібен зовнішній файл. У цій роботі програма читатиме числа з `input.txt`, відбиратиме їх за умовою, обчислюватиме підсумки та записуватиме результат у `output.txt`. Головна ідея — не зберігати все в пам'яті без потреби, а послідовно читати потік, перевіряти відкриття потоків і успішність читання.

2. Очікувані результати

Після виконання роботи ви зможете:

1. Підключити `<fstream>` і створити `std::ifstream` та `std::ofstream`;
2. Перевірити відкриття файла та коректно завершити програму при помилці;
3. Читати числа через `while (input >> value)` і пояснити, чому не використовують `while (!input.eof());`;
4. Записувати результат через `<<` та за потреби читати рядки через `std::getline`;
5. Обробити порожній файл без ділення на нуль і пояснити роль робочого каталогу.

3. Що потрібно знати перед початком

Потрібні змінні, умови, цикли, функції, рядки та базове введення-виведення. Посилання й вказівники не потрібні для основного рішення.

4. Теоретичні відомості

Потоки файлів

`std::ifstream` — потік для читання з файла, `std::ofstream` — потік для запису у файл. Потік є об'єктом C++, пов'язаним із файлом, а не самим файлом.

Відкриття та перевірка

Метод `is_open()` повідомляє, чи потік успішно пов'язаний із файлом. Тому після створення потоку перевіряємо його відкриття до будь-якого читання або запису; для `ofstream` застосовуємо таку саму модель.

```
std::ifstream input("input.txt");
if (!input.is_open())
{
    return 1;
}
```

Якщо вхідний файл не відкрився, обробку продовжувати не можна. Для вихідного файла потрібна така сама перевірка.

Читання чисел

Після операції `input >> value` потік має успішний або неуспішний стан. Умова `while` може перевіряти цей стан без окремого `bool`: тіло циклу виконується лише після того, як чергове значення справді прочитано.

```
int value;
while (input >> value)
{
    // value успішно прочитано
}
```

Умова спочатку намагається прочитати наступне число. Тіло виконується лише після успішного читання; якщо читання не вдалося через кінець файла або некоректні дані, цикл завершується.

Не використовуйте `while (!input.eof())`: кінець файла стає відомим під час невдалої спроби читання, тому перевіряти потрібно саме операцію читання.

Читання рядків і запис

```
std::string line;
while (std::getline(input, line))
{
    output << line << '\n';
}
```

Типове відкриття `ofstream` створює або перезаписує файл. Режим додавання в цій роботі не потрібен.

Робочий каталог і input.txt

Відносний шлях `"input.txt"` не означає «файл поруч із текстом умови». Програма шукає його в робочому каталозі процесу.

Практичний сценарій для ручного запуску:

1. Створіть одну папку `lab10_run`.
2. Покладіть у неї програму `main.cpp` і файл `input.txt`.
3. Відкрийте термінал саме в цій папці; у PowerShell перевірте `Get-Location`, у Linux/macOS — `pwd`.
4. Скомпілюйте `main.cpp` у цій папці й запустіть програму з неї. Тоді `std::ifstream input("input.txt")` шукатиме файл у `lab10_run`.

В IDE відкрийте налаштування запуску й знайдіть **Working Directory / Робочий каталог**; встановіть його на папку, де лежить `input.txt`. Вихідний `output.txt` після запуску також з'явиться в цьому каталозі.

5. Приклади

Приклад 1. Сума чисел із файла

Нехай `input.txt` містить:

```
4 8 -2 0 5 3
```

```
#include <fstream>
#include <iostream>

int main()
{
    std::ifstream input("input.txt");
    if (!input.is_open())
    {
        return 1;
    }
    int value;
    int sum = 0;
    while (input >> value)
    {
        sum += value;
    }
    std::cout << "Sum: " << sum << '\n';
    return 0;
}
```

Програма виведе:

```
Sum: 18
```

Приклад 2. Читання та запис результату

Якщо після попереднього прикладу `sum` дорівнює `18`, вміст `output.txt` буде таким:

```
std::ofstream output("output.txt");
if (!output.is_open())
{
    return 1;
}
output << "sum: " << sum << '\n';
if (!output)
{
    return 1;
}
```

```
sum: 18
```

Приклад 3. Порожній файл

Якщо файл порожній, `count` залишається нульовим. Середне можна обчислювати лише після перевірки `count > 0`; інакше виникне ділення на нуль.

У цьому посібнику для порожнього файлу прийнято таку політику: `count` і `sum` дорівнюють нулю, а замість числового середнього записується `no data`.

```
count: 0
sum: 0
average: no data
```

6. Типові помилки

1. Не перевірено `is_open()`.
2. Після невдалого відкриття програма продовжує обчислення.
3. Використано `while (!input.eof())`.
4. Переплутано вхідний і вихідний потоки.
5. Формат файлу не відповідає очікуваному формату читання.
6. Для порожнього файлу виконано ділення `sum / count`.
7. Відносний шлях шукають не в робочому каталозі процесу.
8. Очікують, що `ofstream` автоматично додаватиме до старого файлу.
9. Використовують абсолютний шлях, прив'язаний до одного комп'ютера.

7. Міні-вправи

Для перевірки непорожнього рядка використовуйте операцію `line.empty()`: вона повертає `true`, якщо рядок порожній. Тому непорожній рядок перевіряють умовою `if (!line.empty())`.

1. Дopiшіть перевірку відкриття `input.txt`.
2. Скільки разів виконається читання для файлу `4 8 2`?
3. Виправте цикл із `!eof()`.
4. Запишіть суму у `output.txt`.
5. Прочитайте рядки та порахуйте непорожні.
6. Знайдіть проблему після змішування `>>` і `getline`.
7. За політикою з прикладу 3 запишіть точний вміст `output.txt` для порожнього `input.txt`. Очікувані три рядки: `count: 0`, `sum: 0`, `average: no data`.

8. Лабораторне завдання

Створіть програму, яка відкриває `input.txt` через `std::ifstream` і перевіряє його відкриття, читає цілі числа до завершення успішного читання, відбирає їх за умовою варіанта, обчислює `count` і `sum`, а потім відкриває `output.txt` через `std::ofstream` та обов'язково перевіряє його відкриття. Якщо вихідний файл не відкрився, завершіть програму з ненульовим кодом і не продовжуйте запис. Якщо `count > 0`, обчисліть `average` як значення типу `double` через дійсне ділення, наприклад `static_cast<double>(sum) / count`; для `count == 0` використайте політику `average: no data`.

Формат результату:

```
count: ...  
sum: ...  
average: ...
```

Для нульової кількості:

```
count: 0  
sum: 0  
average: no data
```

Файл містить не більше 100 коректних цілих чисел, кожне в межах `[-1000, 1000]`, розділених пробілами або новими рядками, або є порожнім. Ці межі є обов'язковою частиною контракту й за технічною моделлю `int` з Додатка А гарантують безпечне накопичення `sum` у `int`; `average` при цьому обчислюється як `double`. Не використовуйте `while (!input.eof())`, двійкове введення-виведення, винятки, `std::vector` або динамічну пам'ять.

9. Індивідуальні варіанти

№	Умова відбору
1	Додатні числа: <code>value > 0</code> .
2	Від'ємні числа: <code>value < 0</code> .
3	Парні числа: <code>value % 2 == 0</code> .
4	Непарні числа: <code>value % 2 != 0</code> .
5	Числа, кратні 3.
6	Числа, кратні 5.
7	Числа, не кратні 4.
8	Числа в <code>[10, 50]</code> .
9	Числа поза <code>[-20, 20]</code> .
10	Числа <code>> 100</code> .
11	Числа <code><= -10</code> .
12	Числа, для яких <code>value % 7 == 1</code> .
13	Числа, кратні 2, але не кратні 4.
14	Числа, кратні 3 або 5.
15	Числа в <code>[-50, -10]</code> .
16	Парні числа в <code>[0, 100]</code> .
17	Числа в <code>[-25, 25]</code> без <code>std::abs</code> .

№	Умова відбору
18	Числа < -100 або > 100 .
19	Числа в $[20, 80]$, не кратні 3.
20	Непарні числа в $[-99, 99]$.

Перевірений спільний приклад

Для `input.txt`:

```
-30 0 1 3 10 21 50 101 105 -101
```

результати для варіантів 1–20 відповідно:

```
1: count 7, sum 291, average 41.5714
2: count 2, sum -131, average -65.5
3: count 4, sum 30, average 7.5
4: count 6, sum 130, average 21.6667
5: count 5, sum 99, average 19.8
6: count 5, sum 135, average 27
7: count 9, sum 160, average 17.7778
8: count 3, sum 81, average 27
9: count 6, sum 146, average 24.3333
10: count 2, sum 206, average 103
11: count 2, sum -131, average -65.5
12: count 2, sum 51, average 25.5
13: count 3, sum 30, average 10
14: count 7, sum 159, average 22.7143
15: count 1, sum -30, average -30
16: count 3, sum 60, average 20
17: count 5, sum 35, average 7
18: count 3, sum 105, average 35
19: count 1, sum 50, average 50
20: count 3, sum 25, average 8.33333
```

Граничні випадки: порожній файл; файл, де жодне число не відповідає умові; файл не існує; значення точно на межі конкретного інтервалу.

10. Підготовка до захисту

Підготуйтеся пояснити `while (input >> value)`, проблему `!eof()`, обробку `count == 0`, різницю між `ifstream` і `cin`, між `ofstream` і `cout`, а також перевірку відкриття файла. Обов'язково вручну пройдіть 3–4 спроби читання заданого файла, спрогнозуйте для них `count`, `sum`, `average` і вміст `output.txt`, а потім внесіть малу зміну в умову відбору або формат результату та повторно спрогнозуйте результат.

11. Контрольні запитання

1. Для чого потрібен `<fstream>`?
2. Чим `ifstream` відрізняється від `cin`?
3. Чим `ofstream` відрізняється від `cout`?
4. Навіщо перевіряти `is_open()`?

5. Що перевіряє `while (input >> value)`?
6. Чому `while (!input.eof())` небезпечний?
7. Що станеться, якщо у файлі замість числа буде текст?
8. Коли використовують `getline(input, line)`?
9. Чому перед середнім треба перевірити `count`?
10. Від чого залежить відносний шлях?
11. Що робить типовий `ofstream` із попереднім вмістом?
12. Чому не обов'язково зберігати всі числа в масиві?

Додаток А. Встановлення та перевірка середовища розробки

Підготувати робоче середовище для лабораторних і перевірити модель `.cpp` → `compiler` → `executable` → `run`.

Редактор, IDE і компілятор

Редактор змінює текст. IDE об'єднує редактор і додаткові інструменти. Компілятор перетворює `.cpp` на програму, яку можна запустити. Встановлений редактор не гарантує, що компілятор уже встановлено.

Лабораторні розраховані на звичайне сучасне середовище GCC, Clang або MSVC із підтримкою C++17. Якщо використовується рекомендоване середовище, окремо перевіряти ширину `int` зазвичай не потрібно.

Усі команди нижче явно задають режим C++17 і не покладаються на розширення, специфічні для конкретного компілятора. Новіший стандарт не є необхідним для виконання лабораторних.

Редакційна примітка: інструкції середовища та команди перевірено станом на 25 серпня 2026 року. Назви пакетів, версії toolchain і елементи інтерфейсу можуть змінюватися, тому за потреби звіряйтеся з офіційними джерелами в кінцевому розділі посібника.

Три поняття для першого запуску

Термінал — вікно, у якому вводять команди. Поточний каталог — папка, з якої команда читає файли й створює результат. Перехід до каталогу змінює лише поточне місце роботи терміналу. Якщо програма відкриває файл відносним шляхом, наприклад `input.txt`, вона шукає його саме в робочому каталозі процесу.

Перевірка компілятора

Створіть файл `hello.cpp`:

```
#include <iostream>

int main()
{
    std::cout << "C++ environment is ready.\n";
    return 0;
}
```

Windows

Рекомендований маршрут для Windows — Visual Studio Community з MSVC. MinGW-w64 залишено нижче як альтернативу. Не змішуйте команди різних toolchain: Visual Studio з MSVC і MinGW-w64 з GCC — це різні компілятори.

Рекомендований маршрут: Visual Studio Community + MSVC

1. Встановіть Visual Studio Community з workload **Desktop development with C++**.
2. Створіть `hello.cpp` у папці, доступній для запису.
3. Відкрийте **Developer Command Prompt for Visual Studio** або **x64 Native Tools Command Prompt**.
4. Перейдіть до папки з файлом і перевірте MSVC командою:

```
cl
```

У виведенні має бути версія Microsoft C/C++ Optimizing Compiler. Скомпілюйте програму з явним режимом C++17 і попередженнями:

```
cl /std:c++17 /W4 /EHsc hello.cpp /Fe:hello.exe
```

Запуск:

```
hello.exe
```

В IDE для того самого режиму виберіть для проєкту C++ Language Standard = **ISO C++17 Standard (/std:c++17)** і Warning Level = **Level 4 (/W4)**, а потім виконайте **Build** → **Build Solution**.

Альтернатива: GCC / MinGW-w64

MinGW-w64 — це toolchain, а не один універсальний інсталятор. Оберіть готовий distribution і після встановлення перевірте, що каталог із `g++` додано до **PATH**.

1. Створіть папку, наприклад `C:\cpp\lab01`, і збережіть у ній `hello.cpp`.
2. Відкрийте PowerShell або термінал обраного MinGW-w64 distribution у цій папці.
3. Перевірте GCC:

```
g++ --version
```

4. Скомпілюйте програму в ISO C++17:

```
g++ -std=c++17 -Wall -Wextra -pedantic hello.cpp -o hello.exe
```

5. У PowerShell запустіть її так:

```
.\hello.exe
```

Linux

У Debian/Ubuntu типовий набір можна встановити через пакетний менеджер:

```
sudo apt update
sudo apt install build-essential
```

Після цього:

```
g++ --version
```

В інших дистрибутивах назви пакетів можуть відрізнятися.

Повний маршрут у Linux: виконайте `mkdir -p ~/cpp/lab01`, потім `cd ~/cpp/lab01`, створіть у цій папці `hello.cpp`, скомпілюйте його командою `g++ -std=c++17 -Wall -Wextra -pedantic hello.cpp -o hello` і запустіть `./hello`.

macOS

Встановіть Command Line Tools for Xcode:

```
xcode-select --install
```

Перевірте:

```
clang++ --version
```

Повний маршрут у macOS: виконайте `mkdir -p ~/cpp/lab01`, потім `cd ~/cpp/lab01`, створіть у цій папці `hello.cpp`, скомпілюйте його командою `clang++ -std=c++17 -Wall -Wextra -pedantic hello.cpp -o hello` і запустіть `./hello`.

Компіляція і запуск

Для GCC/MinGW-w64:

```
g++ -std=c++17 -Wall -Wextra -pedantic hello.cpp -o hello
```

Для Clang:

```
clang++ -std=c++17 -Wall -Wextra -pedantic hello.cpp -o hello
```

Для MSVC у Developer Command Prompt:

```
cl /std:c++17 /W4 /EHsc hello.cpp /Fe:hello.exe
```

GCC і Clang зазвичай запускають результат командою `./hello`, а MSVC — `hello.exe`. У Windows GCC-результат також часто має суфікс `.exe`, наприклад `.\hello.exe`.

Технічна примітка

Числові межі в окремих прикладах і лабораторних роботах розраховані на звичайну desktop-платформу GCC, Clang або MSVC, де знаковий тип `int` має щонайменше 32 біти. Сам стандарт C++17 не зобов'язує кожен платформу мати саме таку ширину `int`. Якщо використовується нестандартне середовище, формальний критерій можна перевірити так:

```
#include <iostream>
#include <limits>

int main()
{
    bool supported = std::numeric_limits<int>::max() >= 2147483647;
    std::cout << (supported ? "int model: supported\n" : "int model: unsupported\n");
    return supported ? 0 : 1;
}
```

Очікуваний результат — `int model: supported`. Якщо отримано `unsupported`, не підмінюйте межі самостійно: повідомте викладача або використайте рекомендоване середовище.

Типові проблеми

- **Compiler not found**: компілятор не встановлено або його каталог не додано до PATH;
- Файл має назву `hello.cpp.txt`: перевірте розширення;
- Команда запускається не з тієї директорії: перевірте робочий каталог;
- Показано старий output: після редагування повторіть компіляцію;
- Компілятор повідомляє про помилку: спочатку виправте вихідний код, потім знову скомпілюйте програму;
- IDE має конфігураційну помилку: окремо перевірте `compiler command`;
- Пробіли або нелатинські символи в шляхах іноді створюють практичні проблеми конкретного toolchain, але це не правило мови C++.

Щоб визначити поточний каталог, у PowerShell виконайте `Get-Location`, у Linux/macOS — `pwd`. Файл `input.txt` або `hello.cpp` має лежати в цьому каталозі, якщо програма відкриває його відносним шляхом.

Що надіслати для діагностики

Вкажіть операційну систему, IDE/toolchain, команду компіляції, повний текст першої помилки, поточну назву файла та результат `--version`. Не надсилайте лише скріншот обрізаного повідомлення.

Додаток Б. Системи числення

Сформувати мінімальну модель позиційних систем числення для розуміння двійкового та шістнадцяткового запису.

Це компактна довідка, а не окрема лабораторна робота. Вона потрібна для фундаментального розуміння подання даних; безпосередньо всі лабораторні завдання на неї не спираються.

Основа і вага позиції

У позиційній системі цифра має вагу, яка залежить від її позиції та основи. Для основи b запис $d_2d_1d_0$ означає:

$$d_2 \cdot b^2 + d_1 \cdot b^1 + d_0 \cdot b^0.$$

Десяткова система має основу 10 і цифри 0–9. Двійкова — основу 2 і цифри 0–1. Шістнадцяткова — основу 16 і цифри 0–9, A–F, де $A = 10$, ..., $F = 15$.

Binary → decimal

$$\begin{aligned} 101101_2 &= 1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 \\ &= 32 + 8 + 4 + 1 \\ &= 45_{10} \end{aligned}$$

Decimal → binary

Для 26 послідовно ділимо на 2 і читаємо остачі знизу вгору:

$$\begin{array}{l} 26 / 2 = 13 \text{ остача } 0 \\ 13 / 2 = 6 \text{ остача } 1 \\ 6 / 2 = 3 \text{ остача } 0 \\ 3 / 2 = 1 \text{ остача } 1 \\ 1 / 2 = 0 \text{ остача } 1 \end{array}$$

Отже, $26_{10} = 11010_2$.

Hexadecimal → decimal

$$2F_{16} = 2 \cdot 16^1 + 15 \cdot 16^0 = 32 + 15 = 47_{10}$$

Тут F — цифра зі значенням 15.

Binary ↔ hexadecimal

Одна шістнадцяткова цифра відповідає групі з чотирьох бітів:

$$11010110_2 \rightarrow 1101 \ 0110 \rightarrow D6_{16}$$

Таблиця:

Decimal	Binary	Hex
0	0000	0

Decimal	Binary	Hex
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

Літери C++

```
#include <iostream>

int main()
{
    std::cout << 10 << '\n';
    std::cout << 0b1010 << '\n';
    std::cout << 0xA << '\n';
    return 0;
}
```

Усі три записи представляють значення 10 і виведуть:

```
10
10
10
```

Типові помилки

- Цифра 2 недопустима в binary;
- 10_2 — це два в десятковій системі, а не десять;
- $0x10$ дорівнює 16_{10} ;
- A у hexadecimal означає 10;
- Число та текстовий запис числа — різні поняття;
- У звичайному десятковому записі ведучі нулі не змінюють значення, але в цілочисловому літералі C++ початковий 0 означає вісімковий запис; без спеціальної потреби не додавайте ведучі нулі до цілих літералів;
- При binary $\leftrightarrow$ hexadecimal групують по 4 біти від молодшого краю.

Міні-вправи

1. Чи коректні записи 10102_2 , $2A_{16}$, 19_2 ?
2. Переведіть 1101_2 у decimal.
3. Переведіть 19_{10} у binary.
4. Переведіть $3C_{16}$ у decimal.
5. Переведіть 11110000_2 у hexadecimal.
6. Порівняйте 10_2 і 10_{10} .
7. Яке значення надрукує C++ для $0x10$?
8. Заповніть відповідність для значень 0–15.

Додаток В. Приклад оформлення лабораторної роботи

Нижче наведено приклад звіту з редагованим текстом, кодом і таблицею. Він показує повне виконання лабораторної роботи №1.

Міністерство освіти і науки України

Івано-Франківський національний технічний університет нафти і газу

Кафедра комп'ютерних систем і мереж

Лабораторна робота №1

Середовище розробки та перша програма

Виконав ст. гр. КІ-26-1:

Петренко Петро Петрович

Перевірив: Пашковський Б.В.

м. Івано-Франківськ

2026

Мета роботи: Навчитися створювати найпростішу програму C++, компілювати її, запускати та перевіряти послідовність текстового виведення.

Завдання: У кожній умові виведіть рівно наведені п'ять рядків. Потім виконайте зазначену зміну.

Варіант 1

№	Початкові 5 рядків	Зміна після першого запуску
1	Програма: старт Файл: main.cpp Мова: C++ Вивід: текст Стан: готово	Стан: готово → Стан: перевірено

Хід роботи

1. Код програми

```
#include <iostream>

int main()
{
    std::cout << "Програма: старт\n";
    std::cout << "Файл: main.cpp\n";
    std::cout << "Дія: компіляція\n";
    std::cout << "Результат: успішно\n";
    std::cout << "Стан: готово\n";
    return 0;
}
```

Результат першого запуску:

```
Програма: старт
Файл: main.cpp
Дія: компіляція
Результат: успішно
Стан: готово
```

Задана зміна: замість рядка «Стан: готово» виведіть «Стан: перевірено». Після зміни файл потрібно зберегти, скомпілювати повторно й запустити новий результат.

Результат після зміни:

```
Програма: старт  
Файл: main.cpp  
Дія: компіляція  
Результат: успішно  
Стан: перевірено
```

Для перевірки реакції компілятора тимчасово було пропущено крапку з комою після однієї інструкції `std::cout`. Компілятор показав синтаксичну помилку; крапку з комою повернуто, після чого програма знову скомпілювалася.

Висновок: у роботі створено програму мовою C++, яка виводить п'ять заданих рядків. Після зміни одного рядка результат оновлено повторною компіляцією. Також розглянуто повідомлення компілятора для простої синтаксичної помилки.

Технічні джерела

Цей короткий список потрібен для перевірки стандарту C++17, команд компіляції, інструкцій середовища та сучасної рамки керування ресурсами. Дата технічної звірки посібника: **25 серпня 2026 року**.

1. **WG21, C++17 working draft N4659.** Матеріали комітету ISO C++ для редакції стандарту C++17: <<https://wg21.link/n4659>>.
2. **GCC, C++ Standards Support.** Підтримка редакцій C++ і явний вибір режиму `-std=c++17`: <<https://gcc.gnu.org/projects/cxx-status.html>>.
3. **Microsoft Learn, Compiling a Native C++ Program on the Command Line.** Developer Command Prompt, `cl`, `/W4` та `/EHsc`: <<https://learn.microsoft.com/en-us/cpp/build/walkthrough-compiling-a-native-cpp-program-on-the-command-line?view=msvc-170>>.
4. **Apple Developer Documentation, Installing the Command Line Tools.** Встановлення Command Line Tools через `xcode-select --install`: <<https://developer.apple.com/documentation/xcode/installing-the-command-line-tools>>.
5. **C++ Core Guidelines, R.11.** Пояснення, чому в прикладному коді не слід явно викликати `new` і `delete`: <<https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#r11-avoid-calling-new-and-delete-explicitly>>.
6. **Debian Packages, build-essential.** Склад типового набору інструментів для Debian/Ubuntu-подібних систем: <<https://packages.debian.org/stable/build-essential>>.

Посилання наведено для технічної перевірки, а не як обов'язковий список для читання перед лабораторною роботою. Якщо інтерфейс або назва пакета змінилися, користуйтеся актуальною інструкцією відповідного виробника toolchain.